

Model Checking

- Logiche temporali
- Automi e logiche temporali
- Modelli per sistemi concorrenti:
 - Process Algebra
 - Reti di Petri

Contesto Statico: Queste logiche operano su un singolo, immutabile stato o mondo. Una formula phi viene valutata rispetto a questo singolo stato di cose.
- Valore di Verità Fissato: Un simbolo proposizionale, come "è lunedì", una volta assegnato un valore di verità in quel "mondo" (per esempio, il 30 Novembre 2025), mantiene quel valore per tutta la valutazione. Non c'è un meccanismo intrinseco per modellare il cambiamento nel tempo.

Le logiche di proposizione e di predicato sono la base, ma non bastano

Logiche temporali



- Logiche classiche considerano un unico "mondo" sul quale esprimono proprietà che sono sempre vere o false
 - Esempio: la proposizione "è lunedì" risulta essere o sempre vera o sempre falsa
- Ma i sistemi software sono sistemi "dinamici"
 - Esempio: la proposizione "la variabile x contiene il valore 10" puo' essere a volte vera a volte falsa
- Le logiche temporali permettono di esprimere proprietà temporali:
 - vere in alcuni "mondi", false in altri

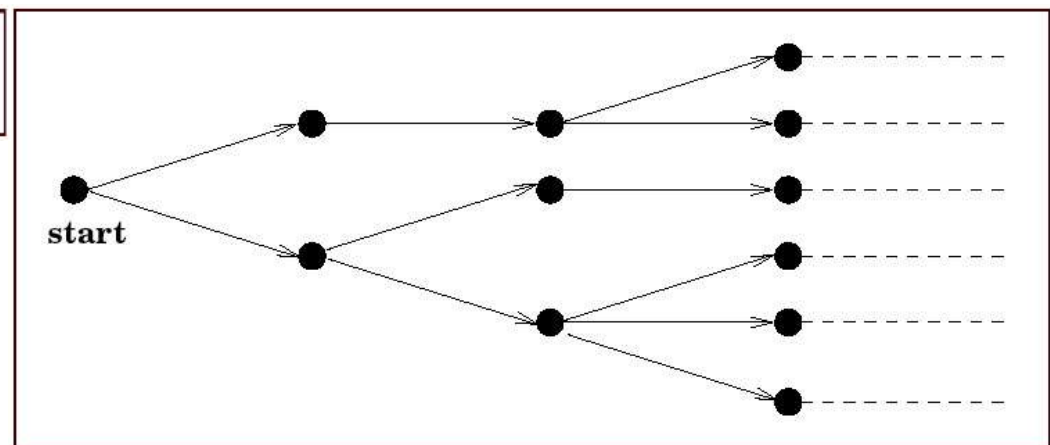
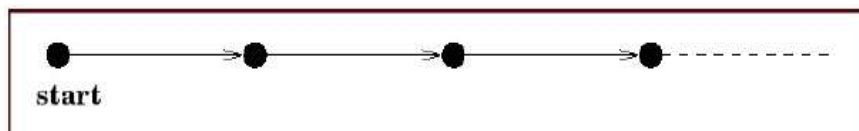
Ho bisogno di aggiungere alle logiche standard degli aspetti dinamici

Le Logiche Temporali sono un'estensione delle logiche classiche che aggiungono operatori modali (i cosiddetti operatori temporali) per esprimere come la verità delle proposizioni cambia nel tempo lungo una sequenza di stati (o "mondi successivi").

Il tempo é rappresentato in modo discreto (a step)

Logiche temporali

- I “mondi” considerati corrispondono a diversi momenti temporali
- Possono esistere diversi modi per mettere in relazione questi “mondi”
 - Linear time:
 - Branching time



Sequenze lineari di "mondi"

- Una sequenza linear time di "mondi" è quindi una sequenza di insiemi di proposizioni che indicano i fatti veri in quell'istante temporale
- Esempi: In ogni istante, ho proposizione vere in quel "mondo", mentre le altre sono false

● → ● → ● → ● → ● → ● ...
{lun} {mar} {mer} {gio} {ven} {sab}

● → ● → ● → ● → ● → ● ...
{s} {s,v} {p} {} {v} {s}
s=sole v=vento p=pioggia

LTL estende la logica proposizionale introducendo gli operatori temporali che ci permettono di ragionare sulle proprietà lungo una singola sequenza lineare di stati (i "mondi" successivi del sistema).

Linear Temporal Logic (LTL)

- Estende la logica proposizionale con operatori per specificare proprietà su sequenze linear time di "mondi":

$\bigcirc \varphi$	φ is true in the <i>next</i> moment in time	Vero nel prossimo "mondo"
$\Box \varphi$	φ is true in <i>all</i> future moments	Vero da ora in poi all'infinito
$\Diamond \varphi$	φ is true in <i>some</i> future moment	Vero in qualche "mondo" futuro
$\varphi \mathcal{U} \psi$	φ is true <i>until</i> ψ is true	psi diventerà vera, e fino ad allora phi è vera.

- Esempio:

$$\Box((\neg passport \vee \neg ticket) \Rightarrow \bigcirc \neg board_flight)$$

Altri esempi

- Sistema in cui si richiede l'esecuzione di un task:
tale richiesta viene ricevuta, elaborata, ed eseguita

$$\Box(requested \Rightarrow \Diamond received)$$

$$\Box(received \Rightarrow \bigcirc processed)$$

$$\Box(processed \Rightarrow \Diamond \Box done)$$

Ci sarà un istante i da cui
sarà per sempre completato

- In tale sistema, se si richiede l'esecuzione del task,
questo sarà sicuramente eseguito. Formalmente la
seguente proprietà, per un tale sistema, è falsa:

$$requested \wedge \Box \neg done$$

Sintassi LTL

$\varphi, \psi \rightarrow$	$p \mid$	(atomic proposition)
	$\top \mid$	(true)
	$\perp \mid$	(false)
	$\neg\varphi \mid$	(complement)
	$\varphi \wedge \psi \mid$	(conjunction)
	$\varphi \vee \psi \mid$	(disjunction)
	$\bigcirc\varphi \mid$	(next time)
	$\Box\varphi \mid$	(always)
	$\Diamond\varphi \mid$	(sometime)
	$\varphi \text{ U } \psi$	(until)

Modelli: sequenze di mondi

- Si considerino **formule di LTL** che utilizzano **proposizioni $p \in P$**

Il Model Checking parte da un insieme di simboli proposizionali P . Queste proposizioni sono gli eventi o fatti base del sistema che possono essere veri o falsi.

- Un **modello** π è una **sequenza infinita** $\pi_1, \pi_2, \pi_3, \dots$ di "mondi" dove π_i è una funzione:

$$\pi_i : P \rightarrow \{T, F\}$$

funzione di valutazione che assegna un valore di verità (Vero o Falso) a tutte le proposizioni atomiche in quell'istante temporale



Nei diagrammi e nei disegni che rappresentano gli stati, spesso non si disegna l'intera funzione (quindi anche le prop. false), ma si elencano solo le proposizioni che sono Vere in quello stato.

- **Nota:** nei disegni visti si rappresenta l'insieme delle proposizioni vere

Semantica di LTL

Proposizione Atomica (p)

- Spiegazione: Una proposizione atomica p è soddisfatta da un cammino π se e solo se p è vera nel primo stato di quel cammino (π_1).

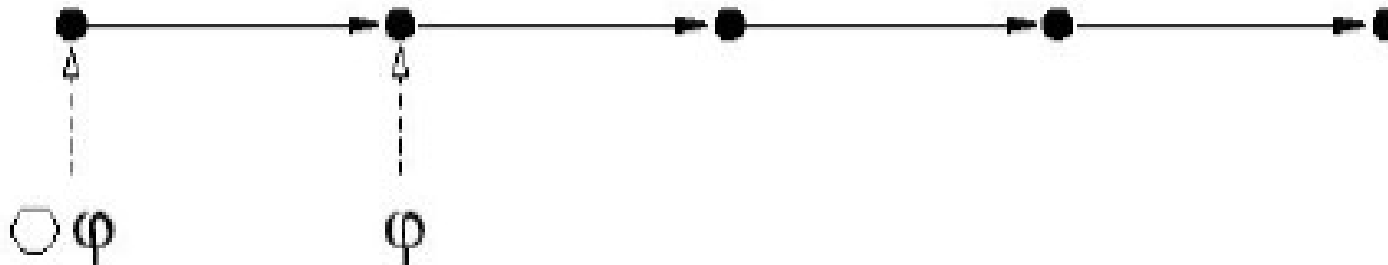
- "Unico diverso da logica proposizionale": Nella logica proposizionale classica, la formula è valutata in un singolo "mondo". Qui, sebbene si valuti solo un singolo stato (π_1), stiamo localizzando la verità all'inizio di una sequenza temporale infinita, che è la novità.

Definizione della relazione $\pi \models \phi$

- $\pi \models p \iff \pi_1(p) = T$ (unico diverso da logica proposizionale)
- $\pi \models \top, \pi \not\models \perp$
- $\pi \models \neg\phi \iff \text{not } \pi \models \phi$
- $\pi \models \phi \wedge \phi' \iff \pi \models \phi \text{ and } \pi \models \phi'$
- $\pi \models \phi \vee \phi' \iff \pi \models \phi \text{ or } \pi \models \phi'$
- $\pi \models \phi \rightarrow \phi' \iff \pi \models \phi \text{ implies } \pi \models \phi'$
- ...

Operatore "next"

- Usato per indicare proposizioni vere nel prossimo momento temporale:



Esempi:

$$(sad \wedge \neg rich) \Rightarrow \bigcirc sad$$

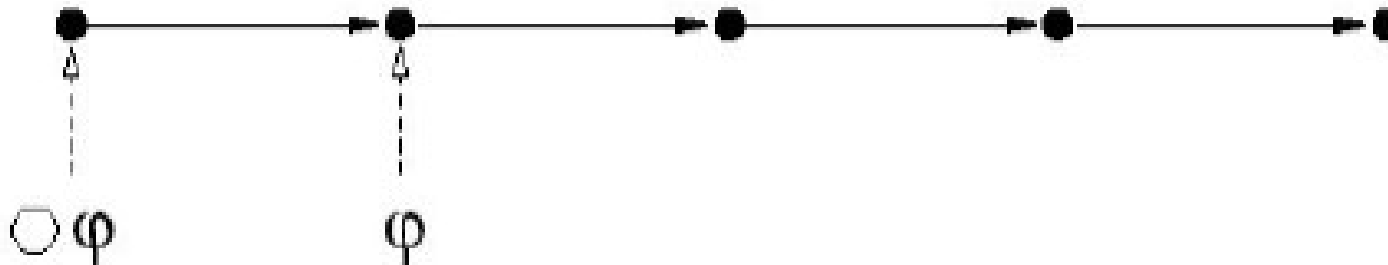
$$((x = 0) \wedge add3) \Rightarrow \bigcirc (x = 3)$$

①

Operatore "next"

L'operatore Next è utilizzato per fare un'affermazione di verità strettamente sul passo temporale immediatamente successivo.

- Usato per indicare proposizioni vere nel prossimo momento temporale:



Semantica:

Sia $\pi = \pi_1, \pi_2, \pi_3, \dots$ il cammino di esecuzione.

Sia π^i la sequenza infinita ottenuta da π a partire dal mondo (stato) i -esimo: $\pi^i = \pi_i, \pi_{i+1}, \pi_{i+2}, \dots$

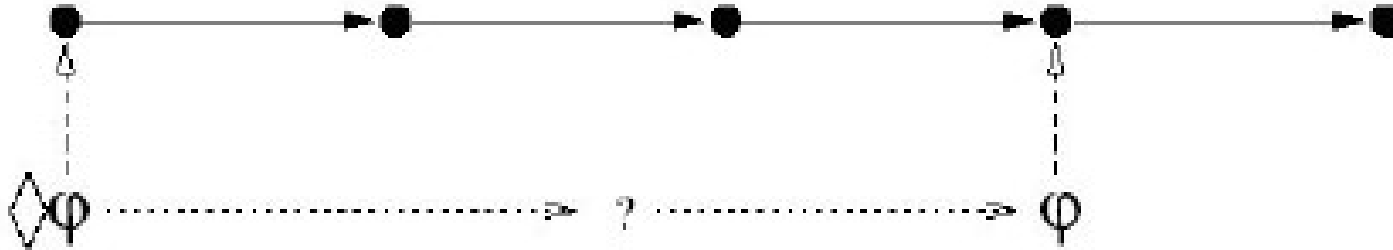
sequenza infinita di mondi che parte dal secondo mondo della sequenza di mondi π

$$\pi \models \bigcirc \varphi \iff \pi^2 \models \varphi$$

Con π^i si rappresenta la sequenza infinita ottenuta da π partendo dal mondo i -esimo

Operatore "sometime"

- Usato per indicare proposizioni vere ora o in un qualche momento futuro:



Esempi:

$$(\neg \text{resigned} \wedge \text{sad}) \Rightarrow \Diamond \text{famous}$$

$$\text{sad} \Rightarrow \Diamond \text{happy}$$

$$\text{send} \Rightarrow \Diamond \text{receive}$$

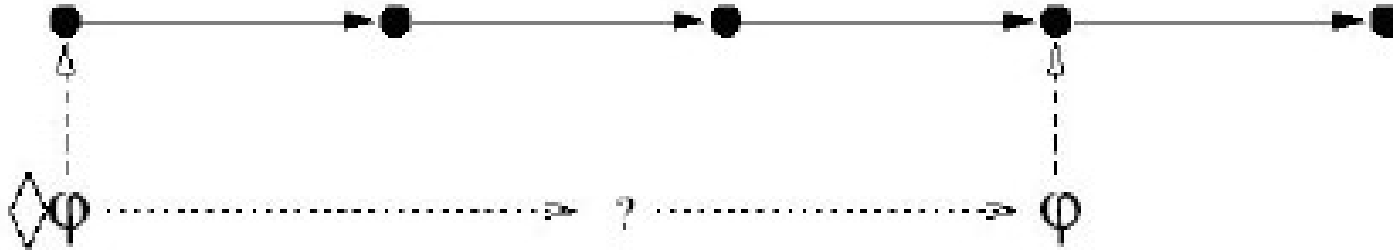
②

Operatore "sometime"



L'operatore Sometime è usato per specificare che una proposizione phi deve diventare vera in un punto del futuro.

- Usato per indicare proposizioni vere ora o in un qualche momento futuro:

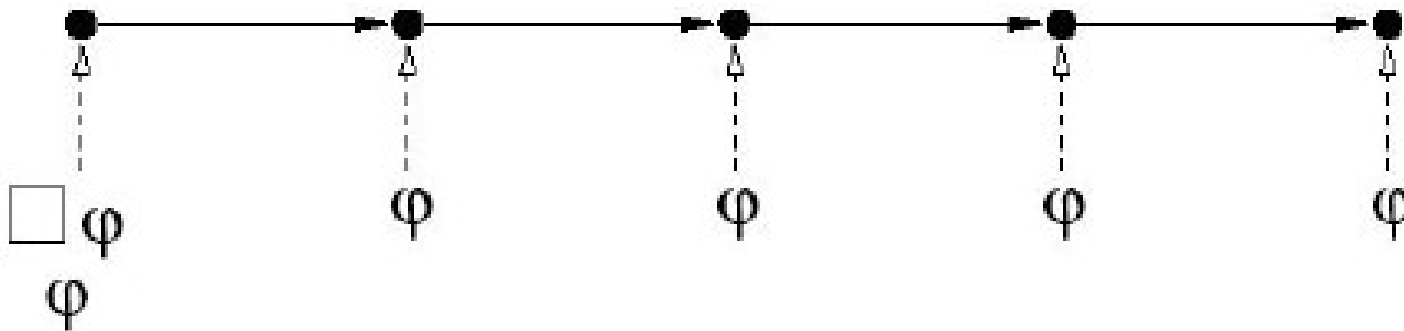


Semantica:

$$\pi \models \Diamond \varphi \iff \text{there is some } i \geq 1 \text{ such that } \pi^i \models \varphi$$

Operatore "always"

- Usato per indicare proposizioni vere ora e in tutti i momenti futuri:



Esempi:

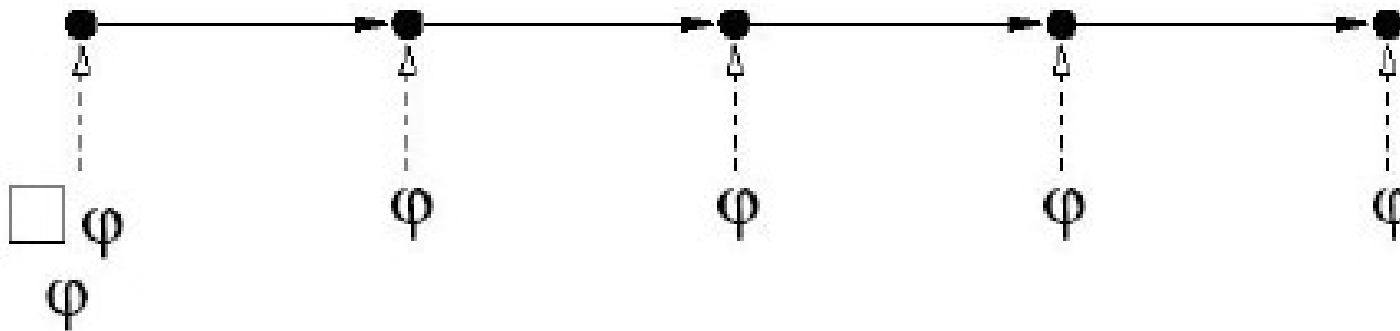
lottery-win $\Rightarrow$ $\square$ *rich*

3

Operatore "always"

- L'operatore Always è usato per specificare che una proposizione ϕ deve essere continuamente vera per tutta l'esecuzione futura del sistema, incluso lo stato corrente.

Usato per indicare proposizioni vere ora e in tutti i momenti futuri:

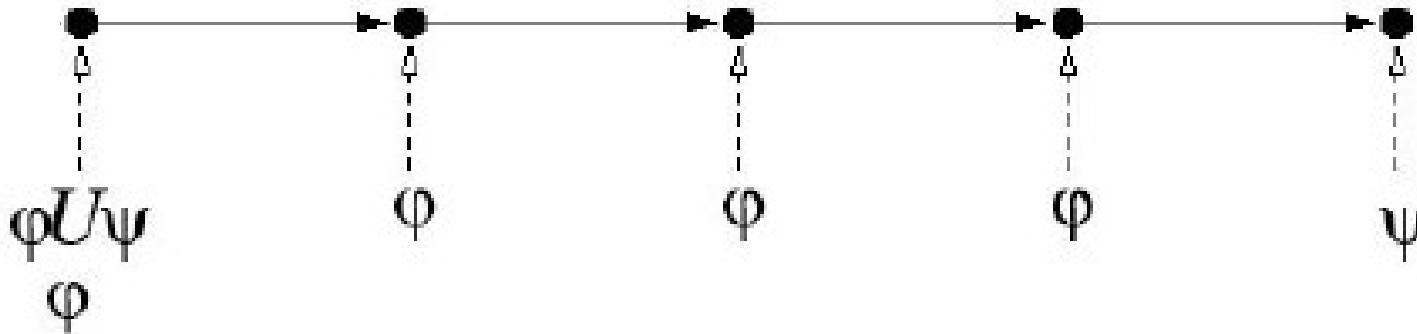


Semantica:

$$\pi \models \Box \phi \iff \text{for all } i \geq 1 \text{ we have } \pi^i \models \phi$$

Operatore “until”

- Usato per mettere in relazione due proposizioni, una vera ora ed in tutti momenti successivi fino ad un momento in cui è vera la seconda:



Esempi:

start_lecture $\Rightarrow$ *talk* $\cup$ *end_lecture*

born $\Rightarrow$ *alive* $\cup$ *dead*

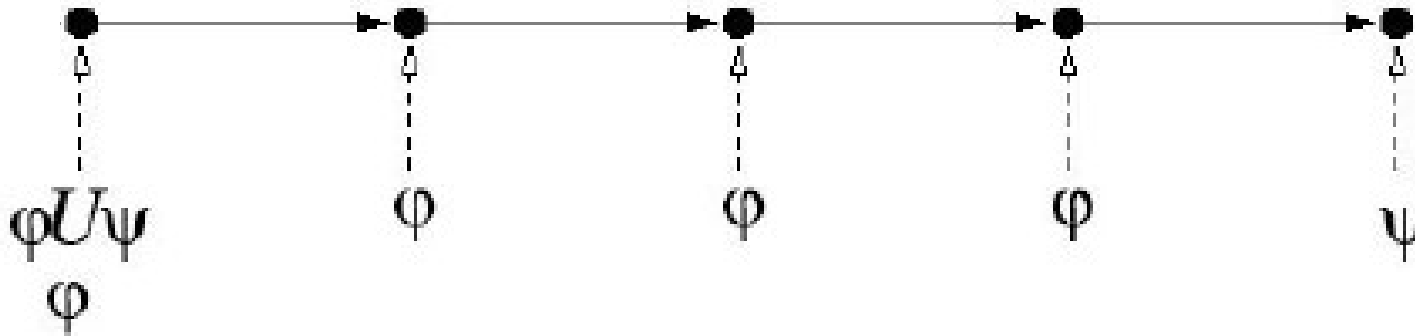
request $\Rightarrow$ *reply* $\cup$ *acknowledgement*

4

Operatore "until"

L'operatore Until è l'operatore LTL più potente perché definisce una relazione di sequenza stretta e limitata tra due proposizioni, phi e psi:
Viene utilizzato per specificare scenari in cui una condizione deve persistere fino al verificarsi di un'altra condizione futura, che è garantita.

- Usato per mettere in relazione due proposizioni, una vera ora ed in tutti momenti successivi fino ad un momento in cui è vera la seconda:



Semantica: psi deve diventare vera prima o poi in un qualche punto del futuro. Questo garantisce che il futuro non sia infinito senza che psi accada.
phi deve essere vera in tutti gli stati precedenti al momento in cui psi diventa vera.

$$\pi \models \varphi u \psi \iff \text{there is some } i \geq 1 \text{ such that } \pi^i \models \psi \text{ and} \\ \text{for all } j = 1 \dots i-1 \text{ we have } \pi^j \models \varphi$$

Operatori minimali

- “always” e “sometime” sono operatori duali:

$$\neg \Box \varphi \equiv \Diamond \neg \varphi$$

- “sometime” può essere espresso usando “until”:

$$\Diamond \varphi \equiv \top \mathcal{U} \varphi$$

- Di conseguenza tutti gli operatori possono essere espressi usando solo “next” e “until”

Relazione con operatori classici

Gli operatori NON distribuiscono in modo incrociato: cioè il sometime non distribuisce con l'AND e always non distribuisce con l'OR

- "sometime" distribuisce su or, "always" su and:

$$\Diamond(\varphi \vee \psi) \equiv \Diamond\varphi \vee \Diamond\psi$$

Se è vero che prima o poi accadrà phi oppure psi, è equivalente a dire che prima o poi accadrà phi OPPURE prima o poi accadrà psi.

$$\Box(\varphi \wedge \psi) \equiv \Box\varphi \wedge \Box\psi$$

Se è vero che sempre accadrà phi AND psi, è equivalente a dire che sempre accadrà phi AND sempre accadrà psi.

- Relazione con il not:

L'operatore next si comporta come un'operazione che non cambia la semantica della negazione, permettendole di "scivolare" all'interno della formula.

$$\neg \bigcirc \varphi \equiv \bigcirc \neg \varphi$$

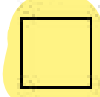
Non è vero che nel prossimo momento phi è vera, è equivalente a dire che nel prossimo momento phi è falsa.

$$\neg(\varphi \mathcal{U} \psi) \equiv (\neg\psi \mathcal{U} (\neg\varphi \wedge \neg\psi)) \vee \Box\neg\psi$$

cioè φ diviene falso prima che ψ sia vero oppure ψ non è mai vero

Notazione alternativa

- Notazione standard testuale (usata da nostro libro) per gli operatori temporali: "U" per "until" e

	$\rightsquigarrow$	<i>F</i>	sometime in the Future
	$\rightsquigarrow$	<i>G</i>	Globally in the future
	$\rightsquigarrow$	<i>X</i>	neXtime

Oppure semplicemente (usata dal nostro tool):

<>	sometime
[]	always
()	next

Proprietà tipiche per sistemi software

■ Esistono utili concetti che tipicamente sono importanti nei sistemi informatici:

- **Proprietà di "safety":**
non si raggiungono mai stati con errori

Le proprietà di Safety affermano che "niente di cattivo accadrà mai". Garantiscono che il sistema rimanga all'interno di un'area operativa sicura.

- Concetto Chiave: Non si raggiungono mai stati con errori.
- Violazione: Una proprietà di Safety è violata se si verifica un evento indesiderato (il "cattivo" evento) anche solo una volta. La violazione è sempre visibile su un prefisso finito del cammino (cioè, un Model Checker può mostrare lo stato di errore dopo un numero finito di passi).

- **Proprietà di "liveness":**
prima o poi si eseguirà una certa azione

Le proprietà di Liveness affermano che "qualcosa di buono accadrà prima o poi". Garantiscono che il sistema progredisca e non si fermi indefinitamente.

- Concetto Chiave: Prima o poi si eseguirà una certa azione.
- Violazione: Una proprietà di Liveness è violata se l'evento atteso non accade mai. La violazione non è visibile su un prefisso finito, ma solo su un cammino infinito che non contiene mai l'evento.

- **Proprietà di "fairness":**
se si richiede una cosa infinite volte, questa verrà eseguita infinite volte

All'infinito, una richiesta deve essere soddisfatta

Le proprietà di Fairness sono un caso speciale di Liveness e sono necessarie per la verifica di sistemi concorrenti e distribuiti. Definiscono condizioni sul non determinismo del sistema, assumendo che i componenti non si comportino in modo costantemente arbitrario o malizioso.

- Concetto Chiave: Se si richiede una cosa infinite volte, questa verrà eseguita infinite volte (o, in generale, se un evento è continuamente abilitato, prima o poi deve essere scelto).

Safety

- “qualcosa di cattivo non accadrà mai”

$$\square \neg (reactor_temp > 1000)$$

- Proprietà di “safety” si esprimono tipicamente così:

$$\square \neg \dots$$

Liveness

- “qualcosa di buono accadrà”

$\diamond rich$

$\diamond (x > 5)$

$\square (start \Rightarrow \diamond terminate)$

$\square (Trying \Rightarrow \diamond Critical)$

- Proprietà di “liveness” si esprimono solitamente così:

$\diamond \dots$

Fairness

- “se qualcosa è richiesta infinite volte, allora avverrà infinite volte”

$$\Box \Diamond ready \Rightarrow \Box \Diamond run$$

Perché uso LTL? LTL è un frammento decidibile della logica dei predicati (esiste un algo che termina sempre nella valutazione di soddisfacibilità)

può essere formalmente vista

LTL come frammento decidibile della logica dei predicati

Una formula LTL ϕ viene codificata come una formula della Logica del Primo Ordine, $\llbracket \phi \rrbracket_z$, dove $\llbracket \phi \rrbracket_t$ è la sua traduzione in un'espressione che dipende dalla variabile temporale t . La traduzione è definita per induzione sulla struttura della formula ϕ .

- Tempo Discreto: Il tempo è modellato dalla variabile t , che assume valori in $\mathbb{N}$.
 - Stato Iniziale: Lo stato iniziale (π_1) è codificato dal numero 0, rappresentato dalla costante z .
 - Transizione: La transizione al prossimo stato (π_{i+1}) è modellata dalla funzione successore $s(t)$. $s(0)=1$, $s(1)=2$, e così via.
 - Proposizioni Atomiche: Una proposizione atomica p è vera nello stato π_i (dove $i \geq 1$) se e solo se il predicato unario $p(i-1)$ è vero.
 $p(t)$ significa " p è vera al tempo t ".

- Le proposizioni $\pi_i(p)$ per $i \geq 1$ sono codificate tramite predicati unari $p(i)$ su numeri naturali
 - costante z associata a 0 e $s(i)$ funzione successore

Una formula LTL φ è codificata dalla formula $\llbracket \varphi \rrbracket_{s(z)}$, dove:

t, t', t'' sono variabili che rappresentano istanti temporali (numeri naturali)

$$\llbracket p \rrbracket_t = p(t) \quad \text{La proposizione } p \text{ è vera all'istante } t.$$

$$\llbracket \bigcirc \varphi \rrbracket_t = \llbracket \varphi \rrbracket_{s(t)} \quad \text{phi è vera nel prossimo istante, che è dato dalla funzione successore di } t.$$

$$\llbracket \Diamond \varphi \rrbracket_t = \exists t' \ t' \geq t \wedge \llbracket \varphi \rrbracket_{t'} \quad \text{phi è vera prima o poi (Finally): esiste un istante } t' \text{ successivo o uguale a } t \text{ in cui phi è vera. (Il tempo } t' \text{ è quantificato).}$$

$$\llbracket \Box \varphi \rrbracket_t = \forall t' \ t' \geq t \rightarrow \llbracket \varphi \rrbracket_{t'} \quad \text{phi è vera sempre (Globally): per ogni istante } t' \text{ successivo o uguale a } t, \text{ phi è vera. (Il tempo } t' \text{ è quantificato universalmente).}$$

$$\llbracket \varphi \cup \psi \rrbracket_t = \exists t' \ t' \geq t \wedge \llbracket \psi \rrbracket_{t'} \wedge \forall t'' \ (t \leq t'' \wedge s(t'') \leq t') \rightarrow \llbracket \varphi \rrbracket_{t''}$$

ψ è vera finalmente (t'), e per tutti gli istanti intermedi (t'') strettamente prima di t' , ϕ è vera.

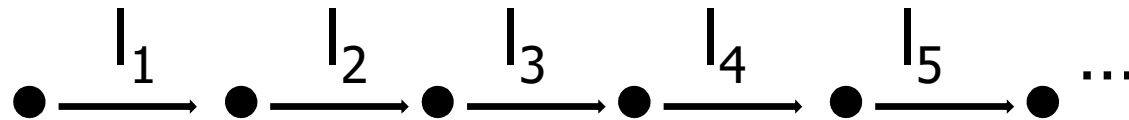
Labeled Transition Systems e Logiche Temporal

Mi interessa il
comportamento del
sistema, non mi interessa
accettare

- Un **Labeled Transition System (LTS)** è un **NFA senza** stati di accettazione **F**

- è una quadrupla (Q, Σ, δ, q_0) con Σ insieme di etichette

- Una **traccia** di un LTS è una sequenza di etichette



- Le tracce ricordano le stringhe degli NFA: iniziano dallo stato iniziale, ma ci interessano le **massimali**

- lunghezza infinita o terminano in uno stato di **blocco** (no transizioni in uscita)

Formalmente...

- Una **traccia** σ di un **LTS** (Q, Σ, δ, q_0) è una sequenza di etichette $l_1 l_2 \dots$ tale che esistono stati $q_1, q_2, \dots$ con $q_i \in \delta(q_{i-1}, l_i)$ per ogni $i = 1 \dots \text{len}(\sigma)$
 - $\text{len}(\sigma)$ è la lunghezza della sequenza (può valere ∞)

q_i appartiene ad un cammino del NFA

- Una **traccia** σ è **massimale** se $\text{len}(\sigma) = \infty$ oppure se $q_{\text{len}(\sigma)}$ è di blocco, cioè $\delta(q_{\text{len}(\sigma)}, l) = \emptyset$ per ogni $l \in \Sigma$

Non ci sono cammini raggiungibili da $q_{\text{len}(\sigma)}$

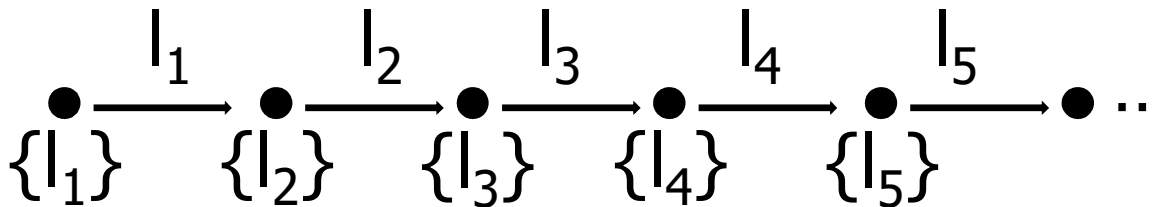
Un Labeled Transition System (LTS) è un formalismo molto comune per modellare il comportamento dei sistemi concorrenti.

- Stati: Nodi che rappresentano gli stati possibili del sistema.
- Transizioni (Archi): Freccie tra gli stati che sono etichettate con le azioni o gli eventi che causano il cambiamento di stato.

Esprimere proprietà di LTS con formule di logica temporale



In ogni stato ("mondo") attraversato lungo una **traccia** vale **una sola proposizione** coincidente con l'**etichetta** della transizione successiva:



Significa che la verifica di una proprietà LTL su un sistema modellato come un LTS deve essere universale e assoluta su tutti i possibili comportamenti del sistema.

Un LTS soddisfa una **formula** di LTL se **tutte le sue tracce massimali** soddisfano tale formula

Formalmente...

- Una **traccia** $\sigma = l_1 l_2 \dots$ di un LTS (Q, Σ, δ, q_0) **individua il modello** π (per formule LTL con proposizioni su Σ) definito da:

$$\pi_i(p) = T \iff i \leq \text{len}(\sigma) \wedge p = l_i$$

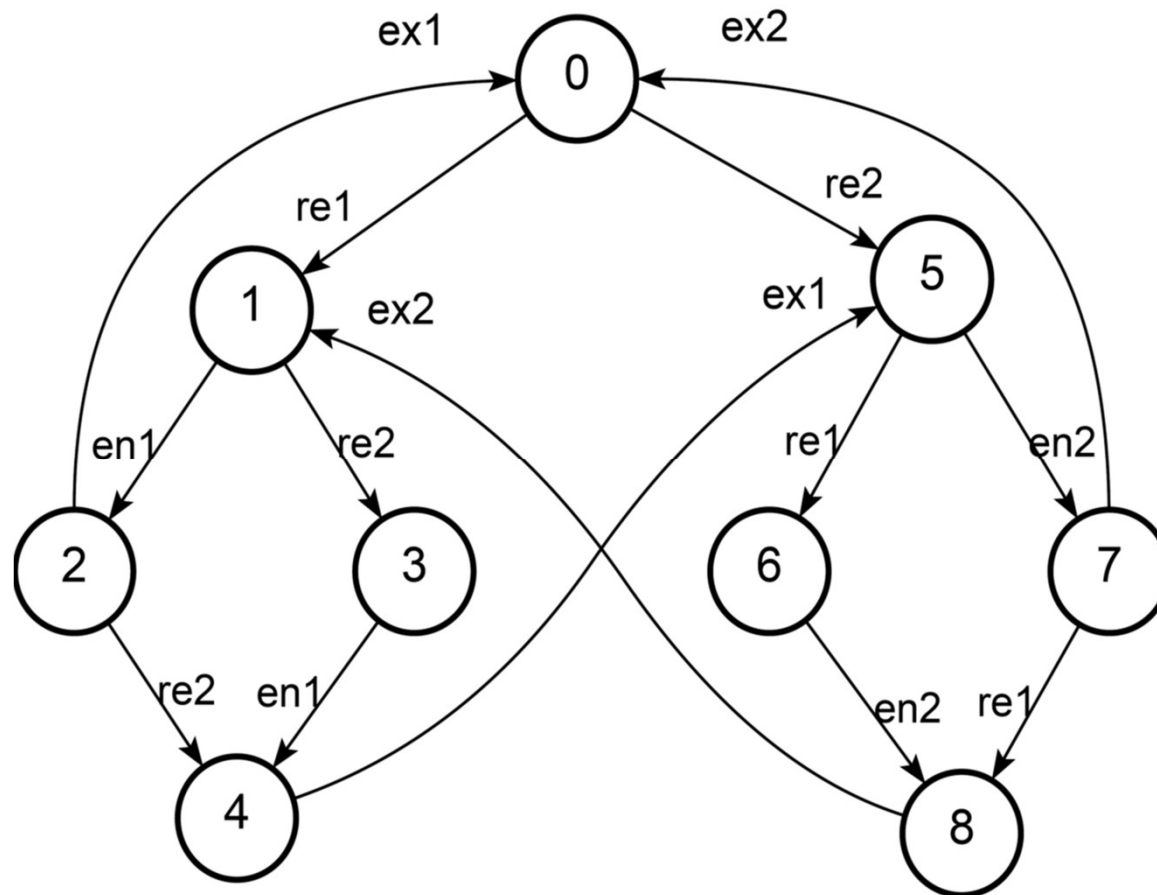
Contesto: Il modello LTL (π) è la sequenza infinita di stati/mondi $\pi_1, \pi_2, \pi_3, \dots$
Significato: La proposizione atomica p è vera nello stato π_i (il mondo all'istante i) se e solo se l'azione p è l' i -esima etichetta l_i nella traccia σ .

dove $i \geq 1$ e $p \in \Sigma$

Se la traccia è finita, le proposizioni dal mondo i in poi (dopo lung. traccia) sono tutte false

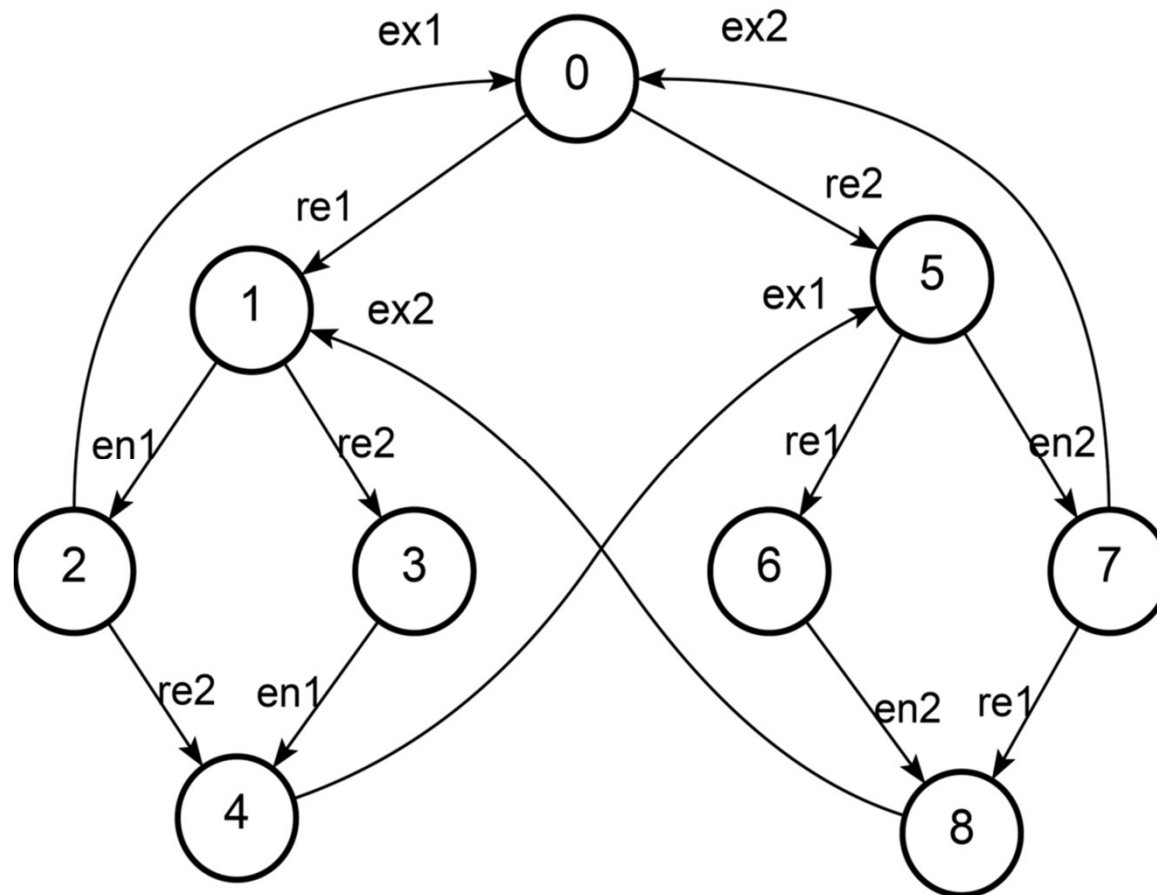
- Un LTS soddisfa una formula ϕ di LTL se i modelli π individuati da **tutte** le sue tracce **massimali** sono tali che: $\pi \models \phi$

Esempio: mutua esclusione



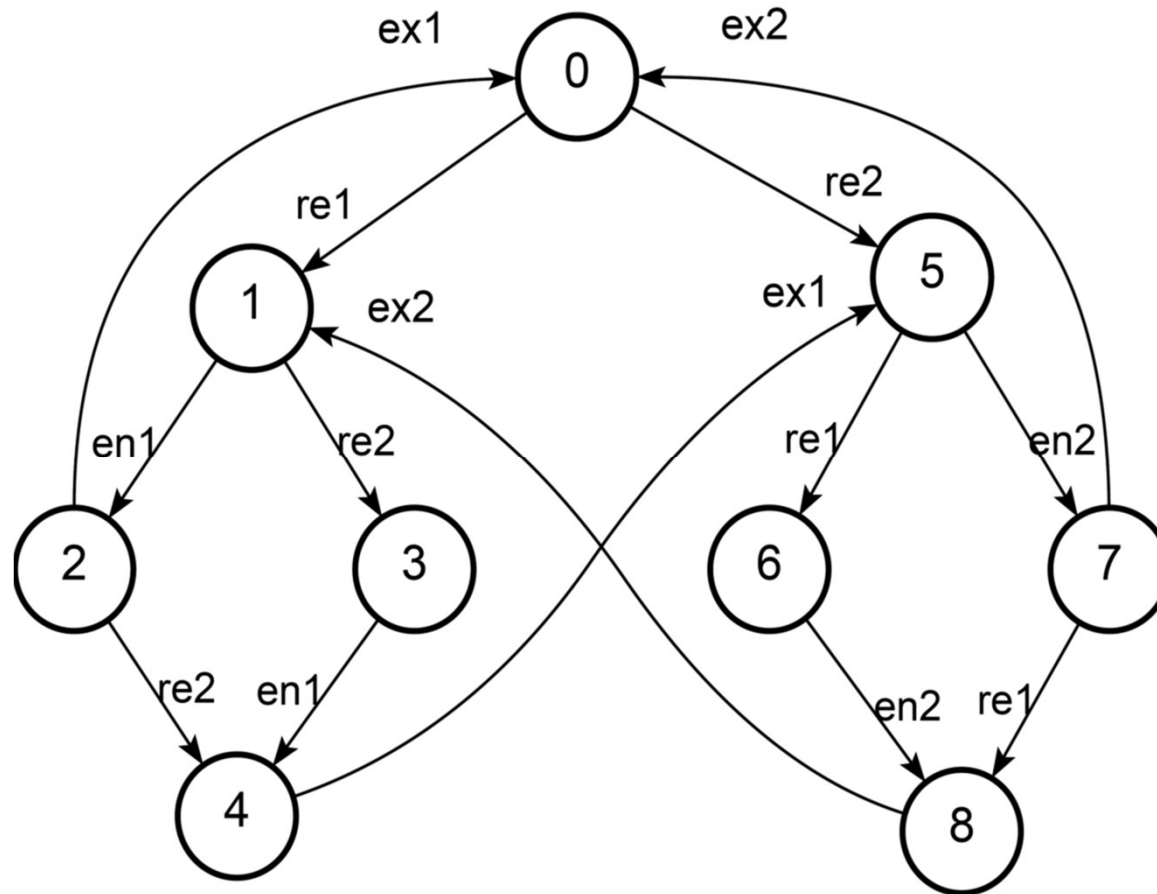
- re1/re2: utente1/utente2 richiedono di entrare in sez. critica
- en1/en2: utente1/utente2 entrano in sezione critica
- ex1/ex2: utente1/utente2 escono da sez. critica

Esempio: mutua esclusione



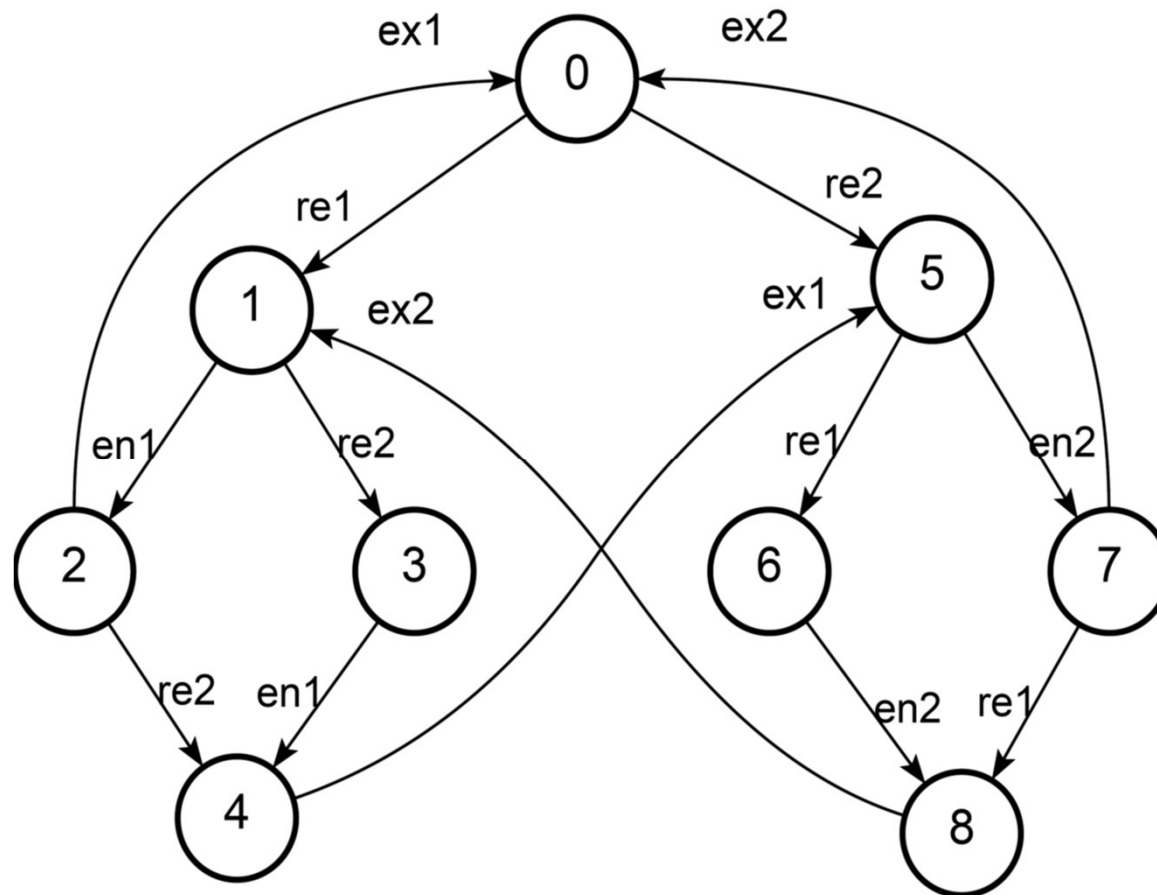
- Proprietà interessanti (safety):
no contemporanea presenza dei due utenti in
sezione critica

Esempio: mutua esclusione



- Proprietà interessanti (safety):
$$[](\text{en1} \Rightarrow (-\text{en2} \cup \text{ex1})) \wedge [](\text{en2} \Rightarrow (-\text{en1} \cup \text{ex2}))$$

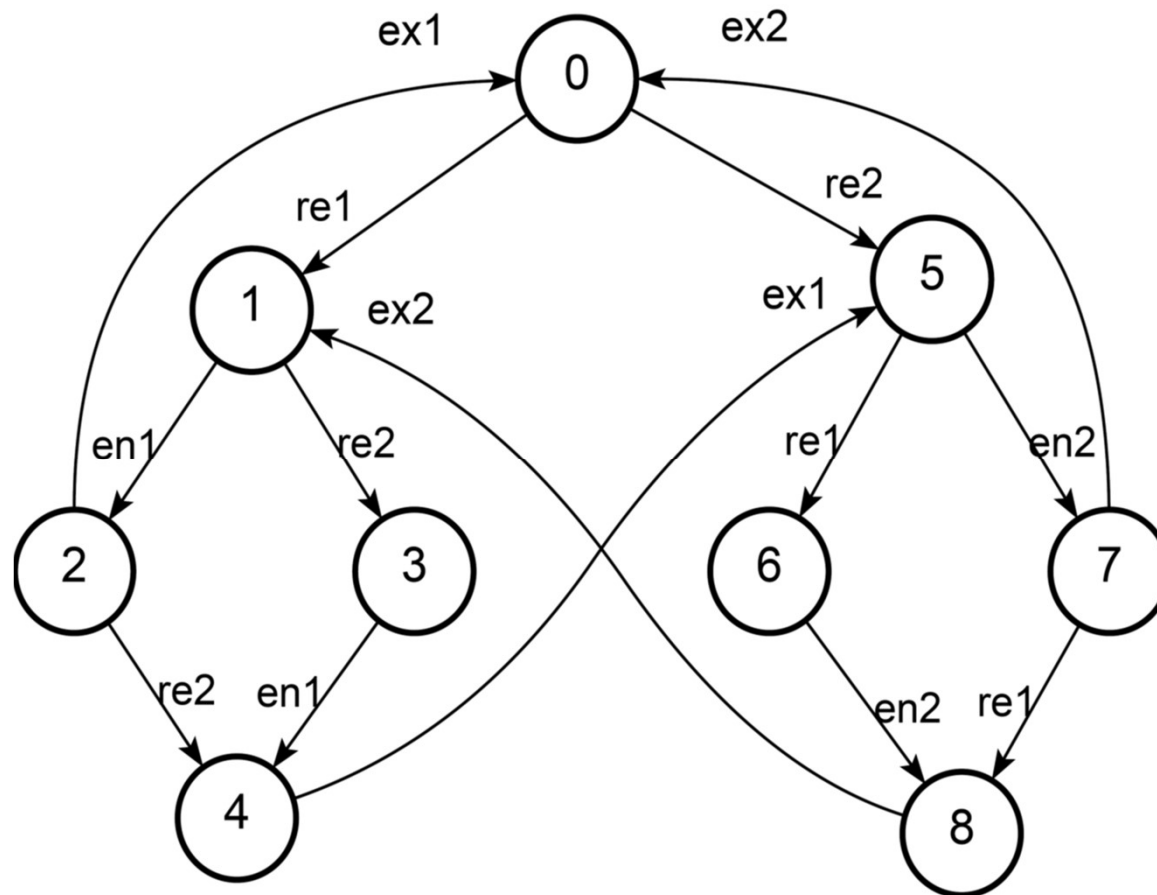
Esempio: mutua esclusione



- Proprietà interessanti (safety):
$$[](\text{en1} \Rightarrow (-\text{en2} \cup \text{ex1})) \wedge [](\text{en2} \Rightarrow (-\text{en1} \cup \text{ex2}))$$

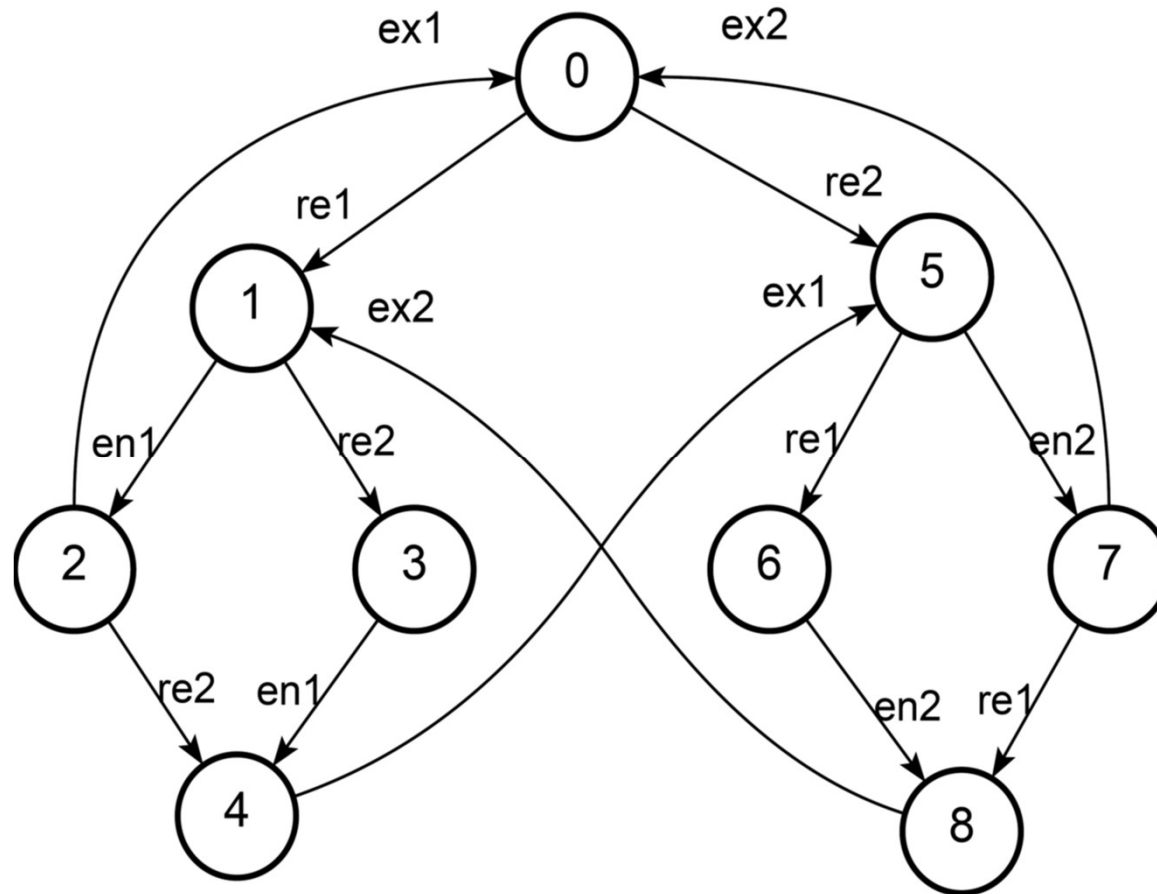
TRUE

Esempio: mutua esclusione



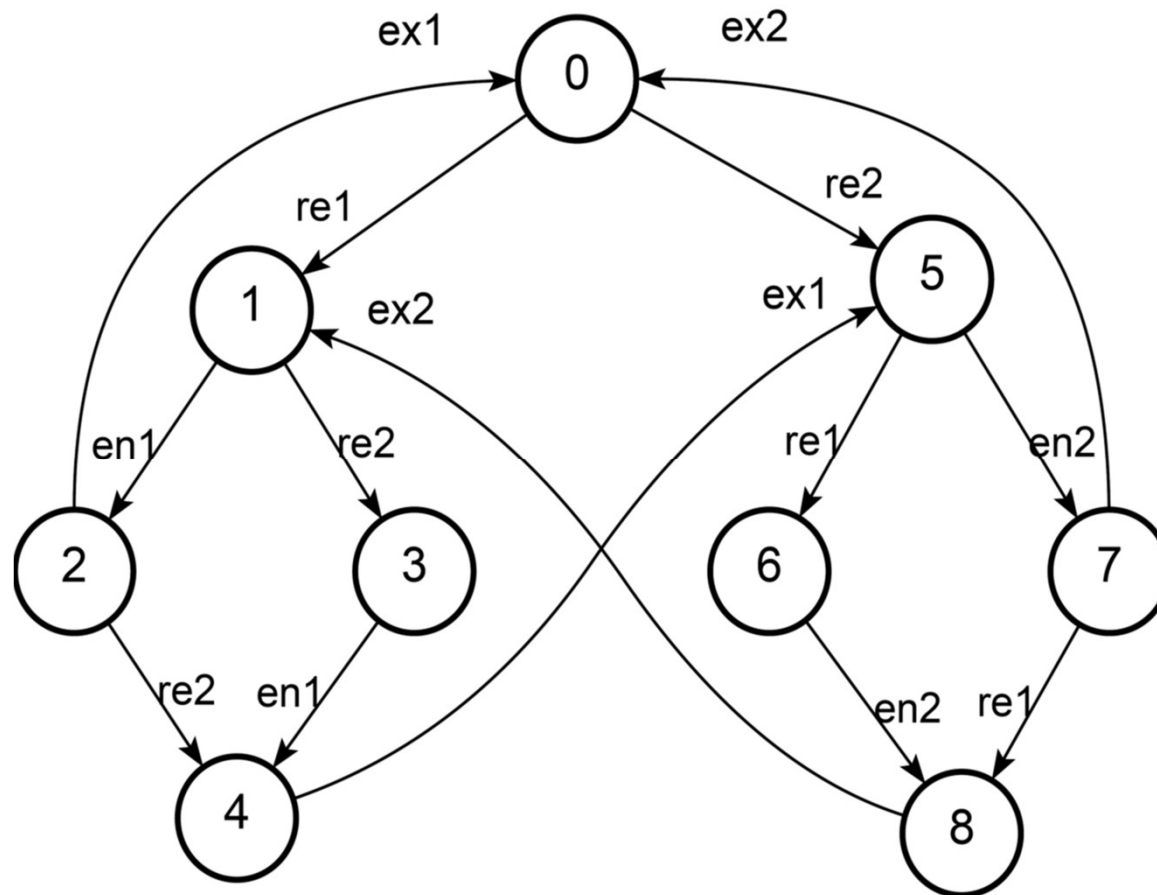
- Proprietà interessanti (liveness):
ogni volta che un utente richiede di entrare,
successivamente entrerà

Esempio: mutua esclusione



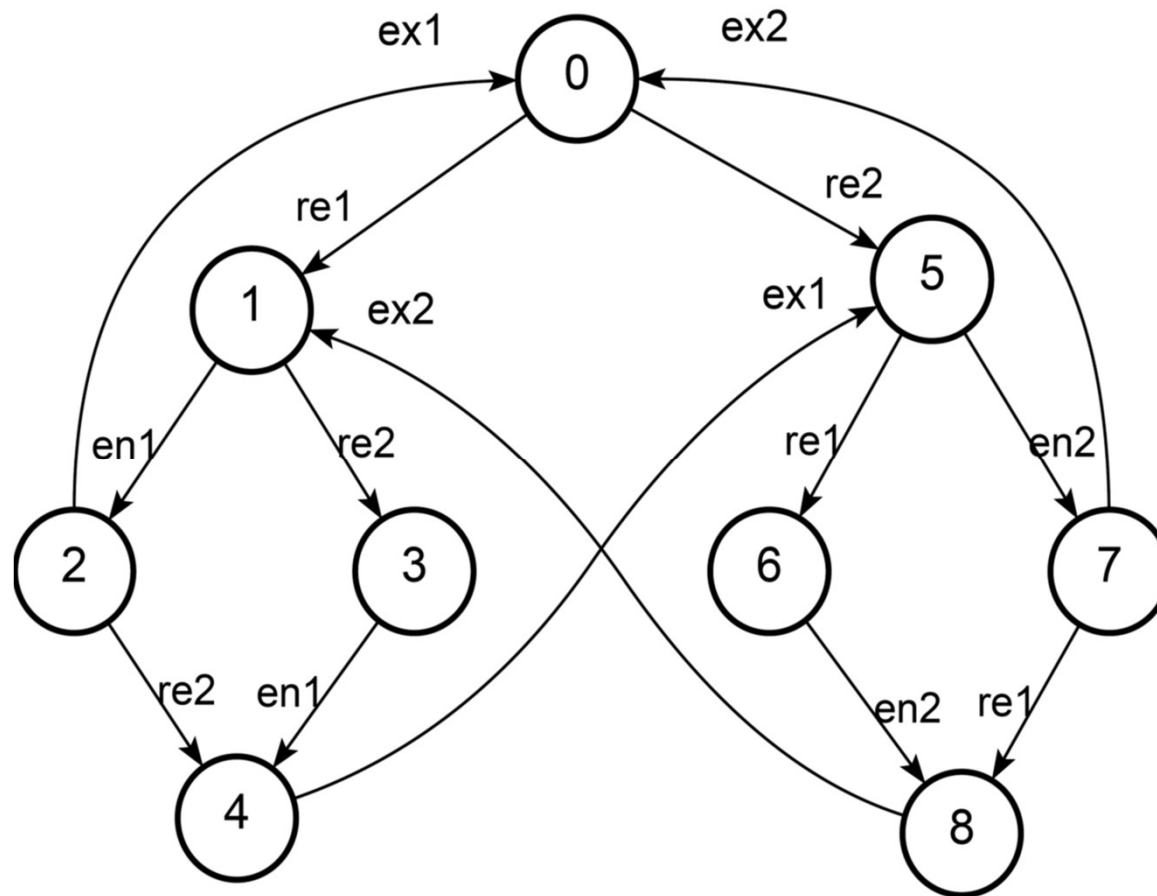
- Proprietà interessanti (liveness):
 $[](\text{re1} \Rightarrow (\langle \rangle \text{en1})) \wedge [](\text{re2} \Rightarrow (\langle \rangle \text{en2}))$

Esempio: mutua esclusione



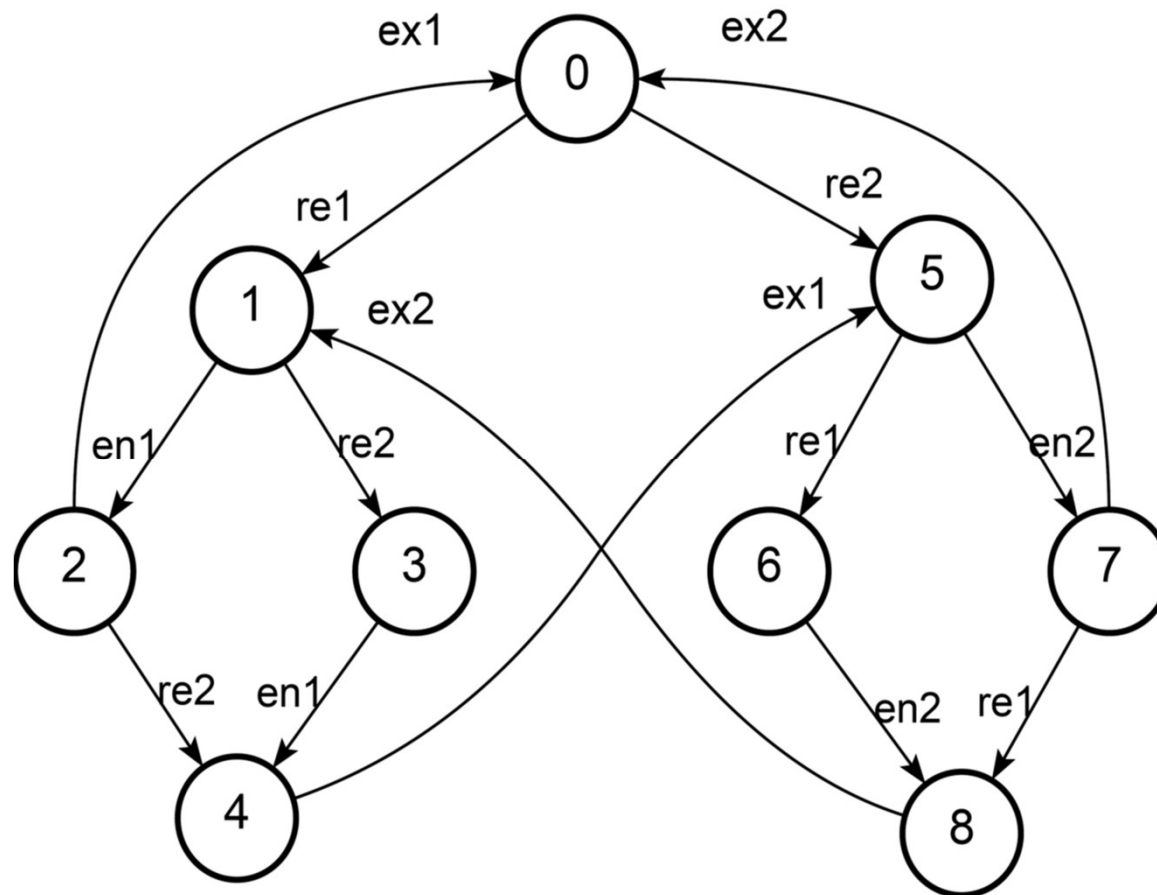
- Proprietà interessanti (liveness):
 $[](\text{re1} \Rightarrow (<>\text{en1})) \wedge [](\text{re2} \Rightarrow (<>\text{en2}))$
TRUE

Esempio: mutua esclusione



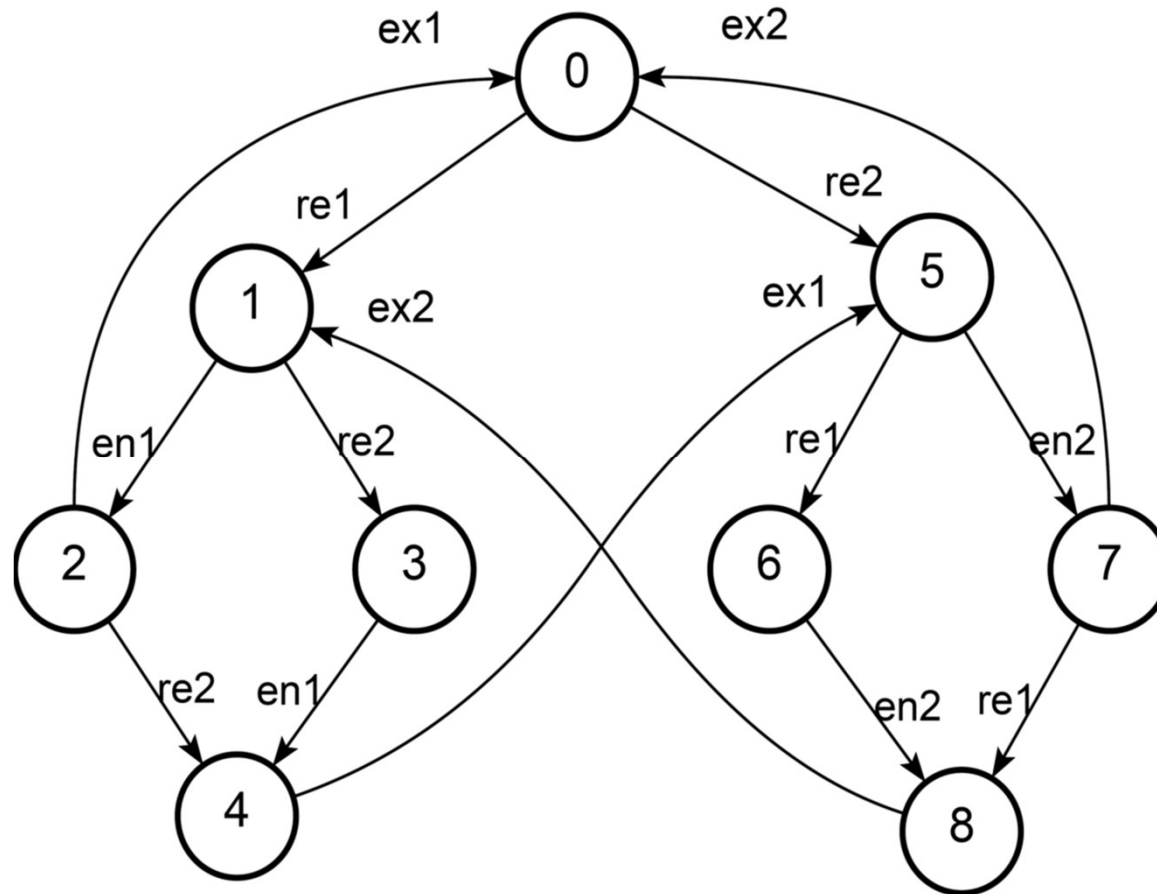
- Proprietà interessanti (fairness):
si noti che si è sempre pronti ad accettare la request di un utente (a meno che non la si abbia già accettata)...

Esempio: mutua esclusione



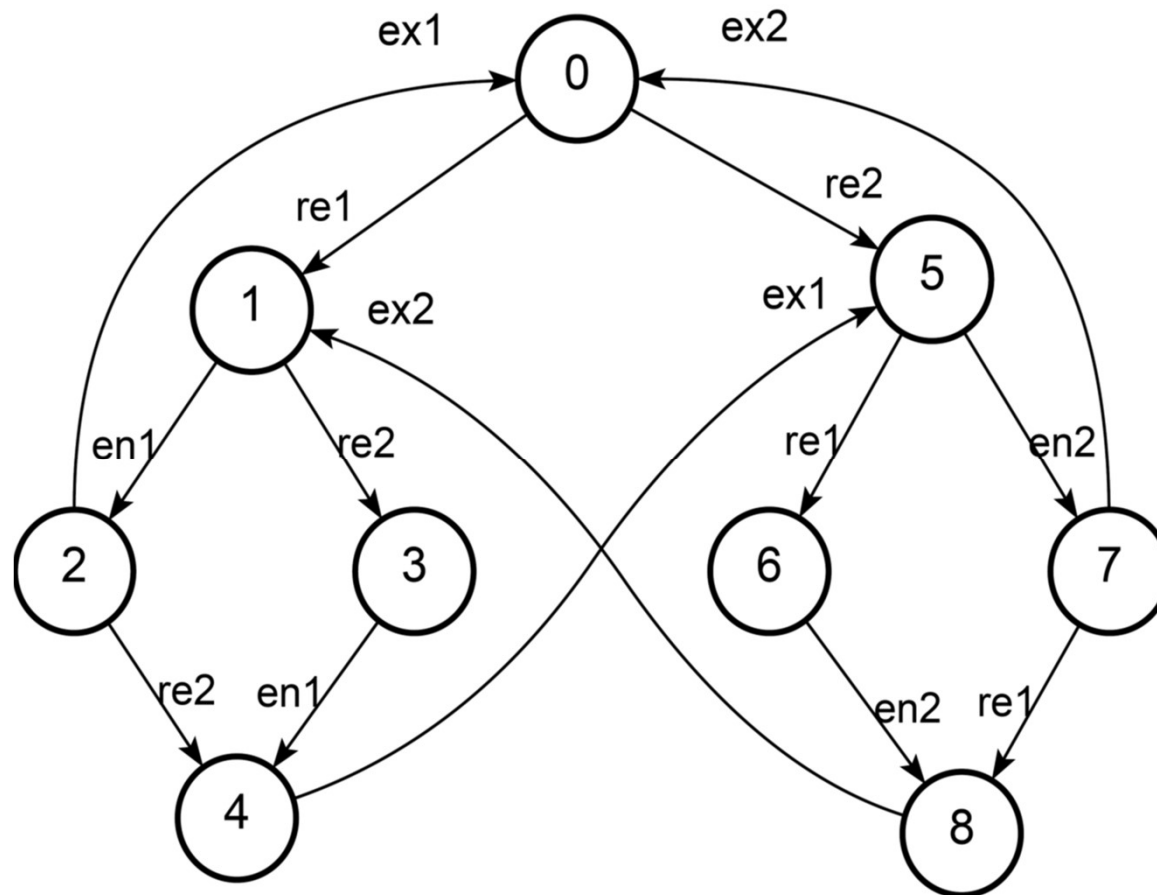
- Proprietà interessanti (fairness):
...ma siamo sicuri che la request verrà effettivamente accettata?

Esempio: mutua esclusione



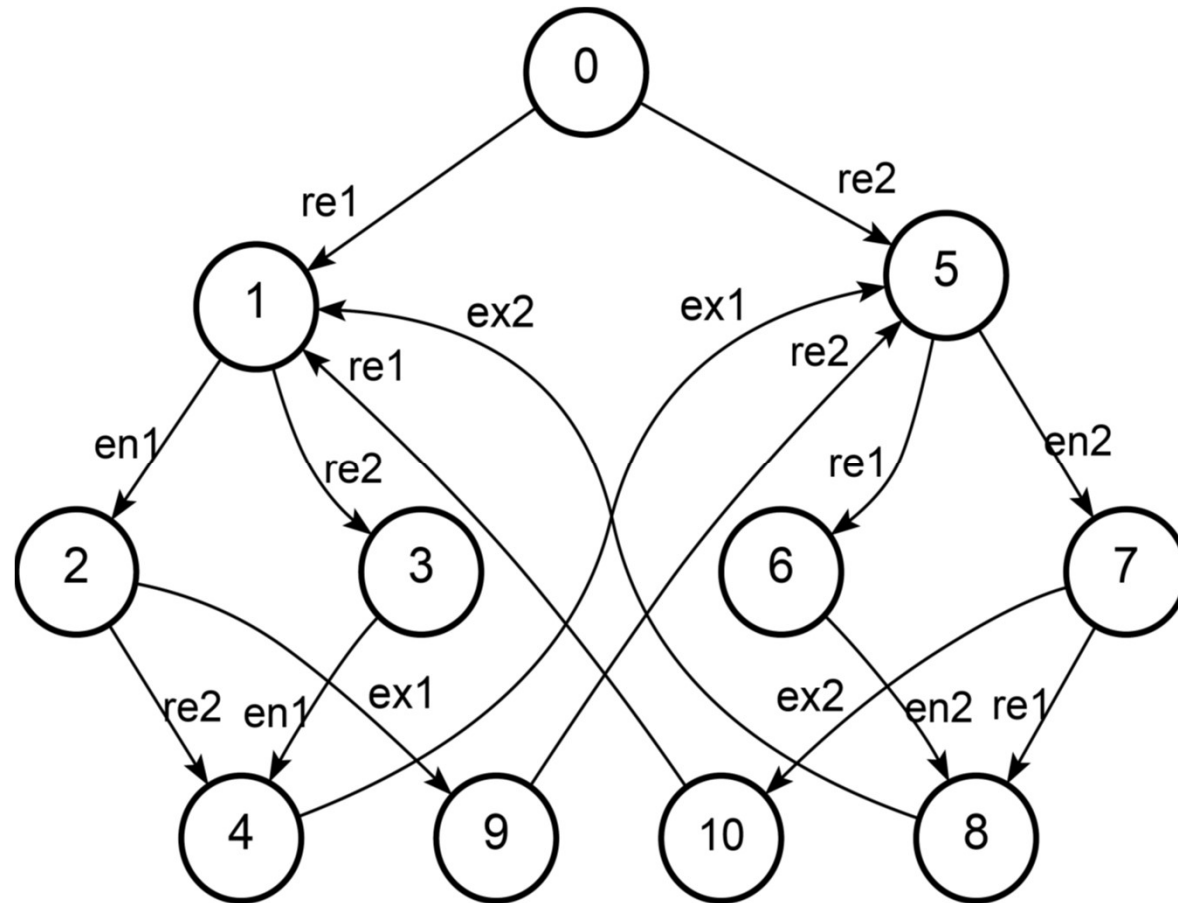
- Proprietà interessanti (fairness):
 $\langle \rangle re1 \wedge \langle \rangle re2$

Esempio: mutua esclusione



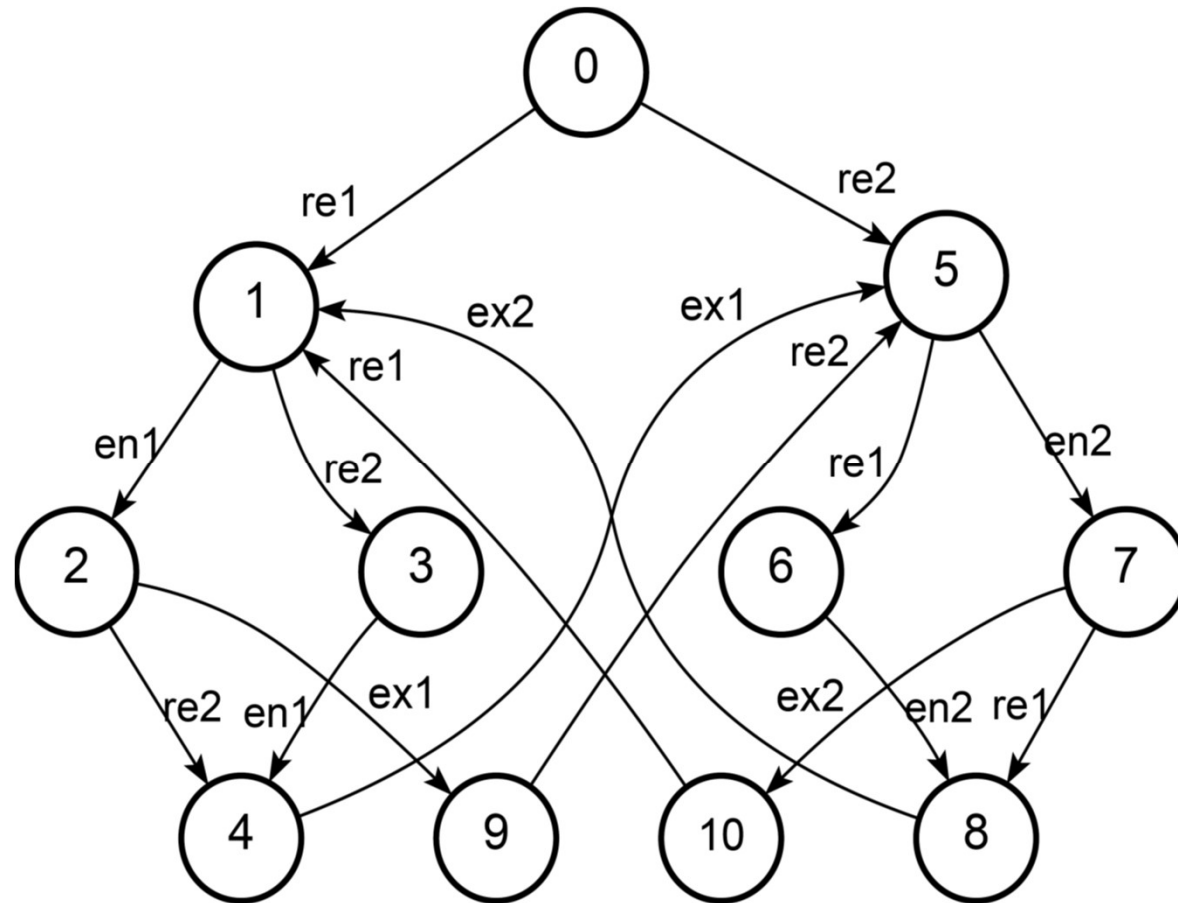
- Proprietà interessanti (fairness):
 $\langle \rangle re1 \wedge \langle \rangle re2$ **FALSE**
(con controesempio $re2-en2-ex2-re2-en2-ex2-\dots$)

Esempio: mutua esclusione (2)



- Nuova versione dell'automa:
forzando l'alternanza fra gli utenti, sarà sempre vero che entrambi potranno eseguire prima o poi la loro request

Esempio: mutua esclusione (2)



- Nuova versione dell'automa:
 $[<>re1 \wedge <>re2]$

TRUE

Sistemi concorrenti

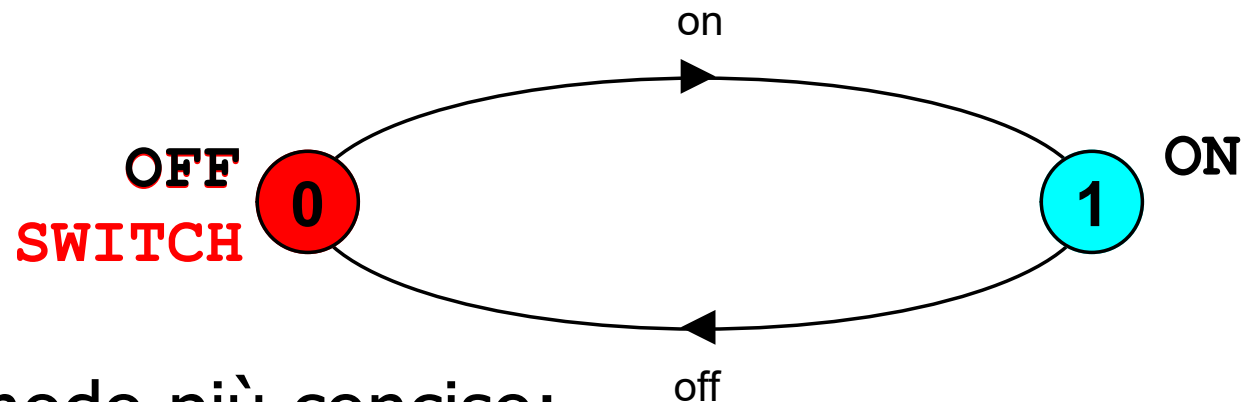
- I sistemi modellati da tali automi (LTS) sono solitamente “sistemi concorrenti”:
 - Vari processi **interagiscono** (es: i due utenti che interagiscono per accedere alla sezione critica)
- Le “process algebra” sono un modo naturale per rappresentare sistemi di questo tipo
 - Descrivere **ogni processo** da solo
 - **Combinare** le descrizioni di tali processi per ottenere l'intero sistema

Process Algebra

- Esistono tanti tipi di process algebra (CCS, ACP, CSP, pi-calculus, ...)
- Trattiamo FSP (Finite State Processes) usato dal tool LTSA (Labeled Transition System Analyzer)
- Un processo è rappresentato come un LTS:
 - si danno **nomi agli stati**
 - per ogni stato si descrivono le **transizioni** che lo stato **può fare**

Esempio: interruttore

SWITCH = **OFF** ,
OFF = (on -> **ON**) ,
ON = (off-> **OFF**) .

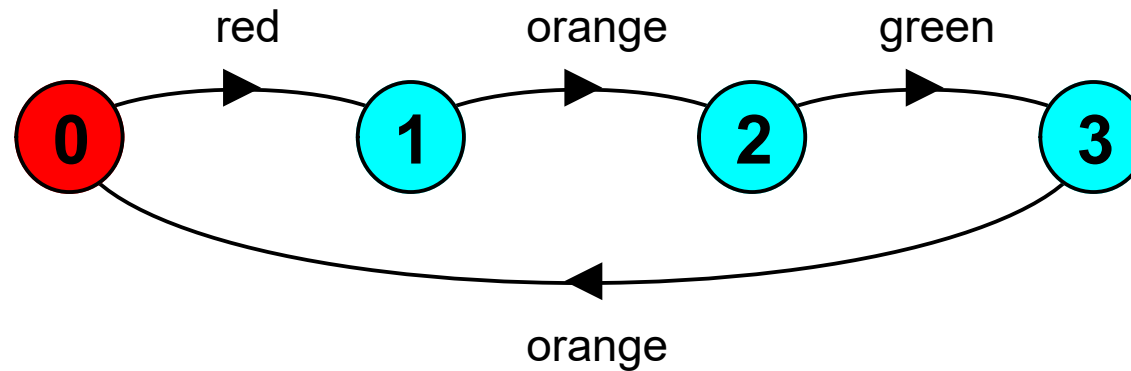


- Oppure, in modo più conciso:

SWITCH = (on->off->**SWITCH**) .

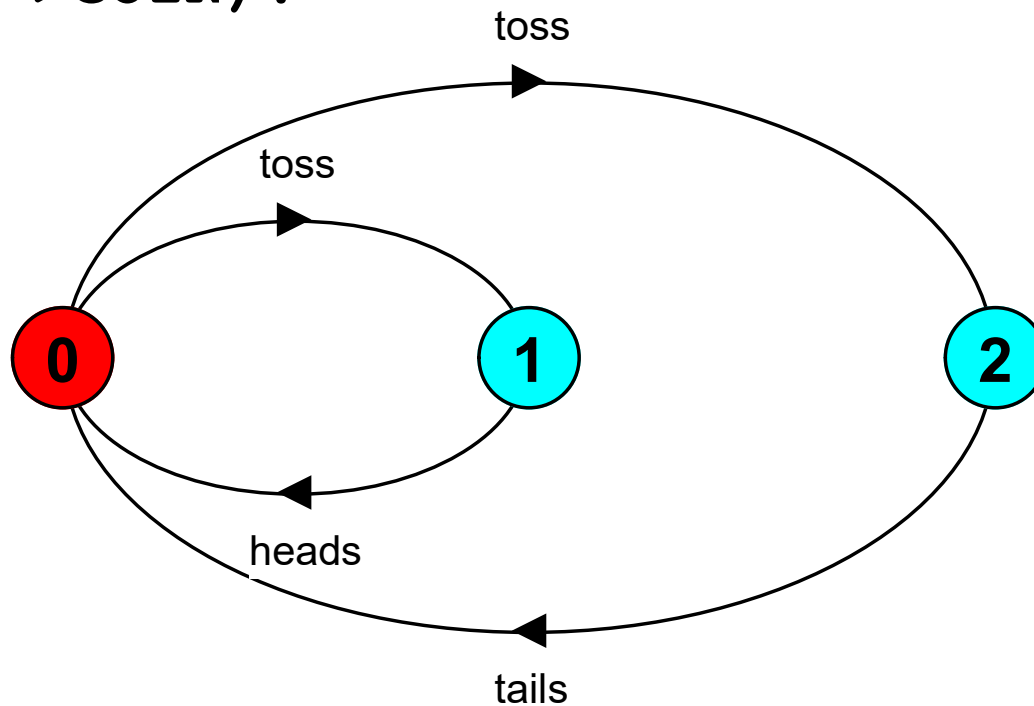
Esempio: semaforo

**TRAFFICLIGHT = (red->orange->green->orange
-> TRAFFICLIGHT) .**



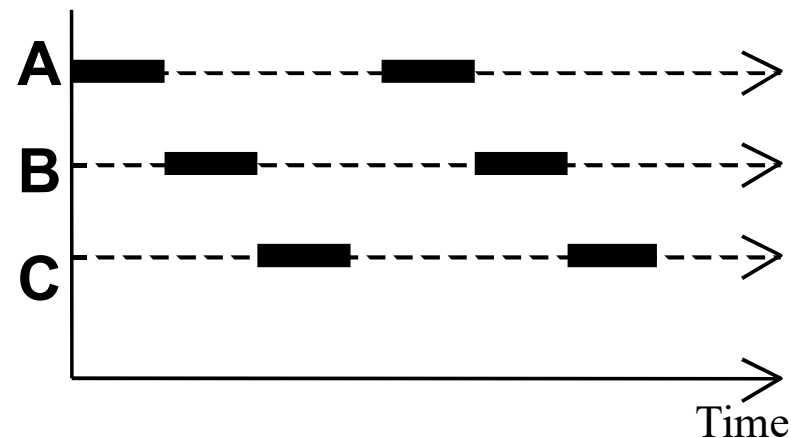
Esempio: lancio della moneta

```
COIN = (toss->HEADS | toss->TAILS) ,  
HEADS= (heads->COIN) ,  
TAILS= (tails->COIN) .
```



Definizioni

- Concorrenza (interleaving):
 - Esecuzione "logicamente" parallela di processi (LTS), come in un sistema multitasking



- Sincronizzazione:
 - Esecuzione "fisicamente" contemporanea di azioni

Parallelismo: interleaving e sincronizzazione

- Interleaving:
 - Transizioni **specifiche** di un processo (cioè assenti in altri processi) vengono eseguite in “interleaving”
 - **Interleaving**: una azione alla volta in **ordine arbitrario** (nessuna assunzione sulla velocità relativa dei processi)
- Sincronizzazione:
 - Su **etichette comuni** (nell’alfabeto degli LTS eseguiti in parallelo) è prevista “sincronizzazione”
 - **Sincronizzazione**: esecuzione **simultanea** di una azione da parte di più LTS

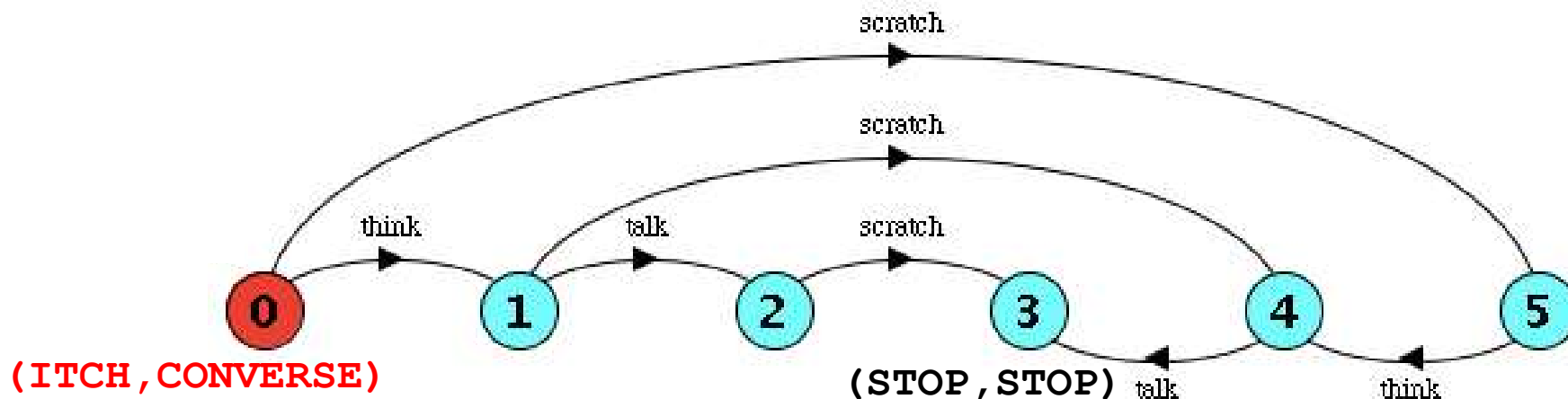
Esempio: composizione parallela tramite interleaving

`ITCH = (scratch->STOP) .`

`CONVERSE = (think->talk->STOP) .`

`|| CONVERSE_ITCH = (ITCH || CONVERSE) .`

- Avendo **alfabeti disgiunti**, le azioni vengono eseguite in interleaving (STOP è stato di blocco):



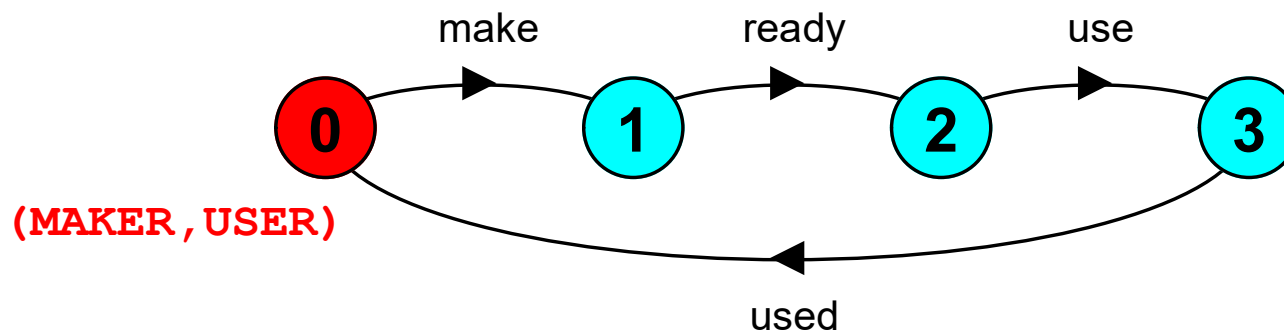
Esempio: composizione parallela con sincronizzazione

MAKER = (make->ready->used->MAKER) .

USER = (ready->use->used->USER) .

||MAKER_USER = (MAKER || USER) .

- “ready” e “used” sono **in entrambi gli alfabeti**, quindi tali transizioni devono **sincronizzarsi**:



Formalmente

Dati due LTS $A_i = (Q_i, \Sigma_i, \delta_i, q_0^i)$, la loro **composizione parallela** $A_1 || A_2$ è l'LTS $(Q_1 \times Q_2, \Sigma_1 \cup \Sigma_2, \delta, (q_0^1, q_0^2))$, dove δ è definita:

- per ogni $a \in \Sigma_1 \cap \Sigma_2$ (caso della **sincronizzazione**)

$$\delta((p, q), a) = \delta_1(p, a) \times \delta_2(q, a)$$

cioè se $p \xrightarrow{a} p'$ e $q \xrightarrow{a} q'$ allora $(p, q) \xrightarrow{a} (p', q')$

- per ogni $a \notin \Sigma_1 \cap \Sigma_2$ (caso dell'**interleaving**)

$$\delta((p, q), a) = \delta_1(p, a) \times \{q\} \quad \text{se } a \in \Sigma_1$$

cioè se $p \xrightarrow{a} p'$ allora $(p, q) \xrightarrow{a} (p', q)$

$$\delta((p, q), a) = \{p\} \times \delta_2(q, a) \quad \text{se } a \in \Sigma_2$$

cioè se $q \xrightarrow{a} q'$ allora $(p, q) \xrightarrow{a} (p, q')$

Proprietà della composizione parallela

- La composizione parallela è **commutativa** e **associativa**
 - L'LTS di $(A_1 || A_2)$ è lo stesso di $(A_2 || A_1)$
 - L'LTS di $(A_1 || A_2) || A_3$ è lo stesso di $A_1 || (A_2 || A_3)$
- Possiamo quindi denotare la **composizione parallela di multipli LTS** $A_1, A_2, \dots, A_n$ scrivendo semplicemente $A_1 || A_2 || \dots || A_n$

Esempio: composizione parallela multipla

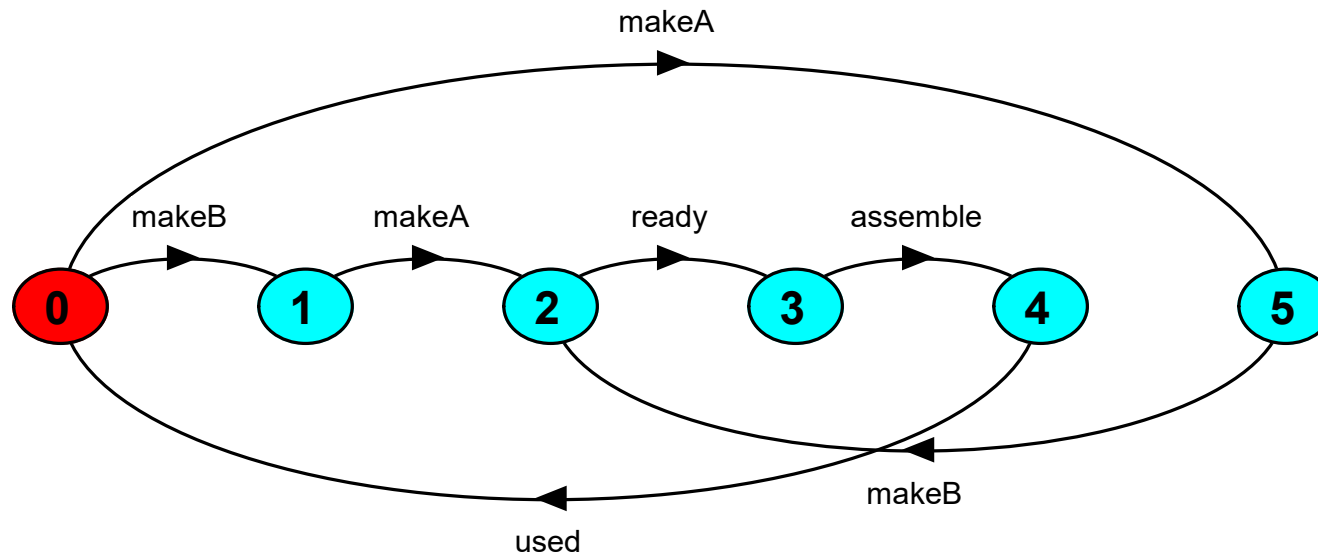
MAKE_A = (makeA->ready->used->MAKE_A) .

MAKE_B = (makeB->ready->used->MAKE_B) .

ASSEMBLE = (ready->assemble->used->ASSEMBLE) .

|| FACTORY = (MAKE_A || MAKE_B || ASSEMBLE) .

- “ready” e “used” sono in tutti e 3 gli alfabeti, quindi i tre processi devono sincronizzarsi:



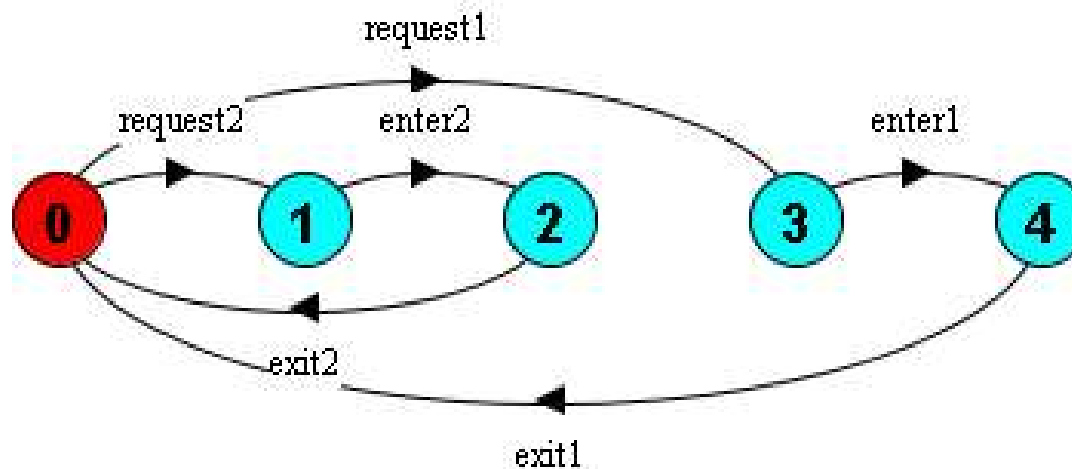
Esempio: mutua esclusione

USER1 = (request1 -> enter1 -> exit1 -> USER1) .

USER2 = (request2 -> enter2 -> exit2 -> USER2) .

CONTROLLER = (request1->enter1->exit1->CONTROLLER |
request2->enter2->exit2->CONTROLLER) .

||MUTUAL_EXCLUSION = (USER1 || USER2 || CONTROLLER) .



Esempio: mutua esclusione (2)

```
USER1 = (request1 -> enter1 -> exit1 -> USER1) .  
USER2 = (request2 -> enter2 -> exit2 -> USER2) .
```

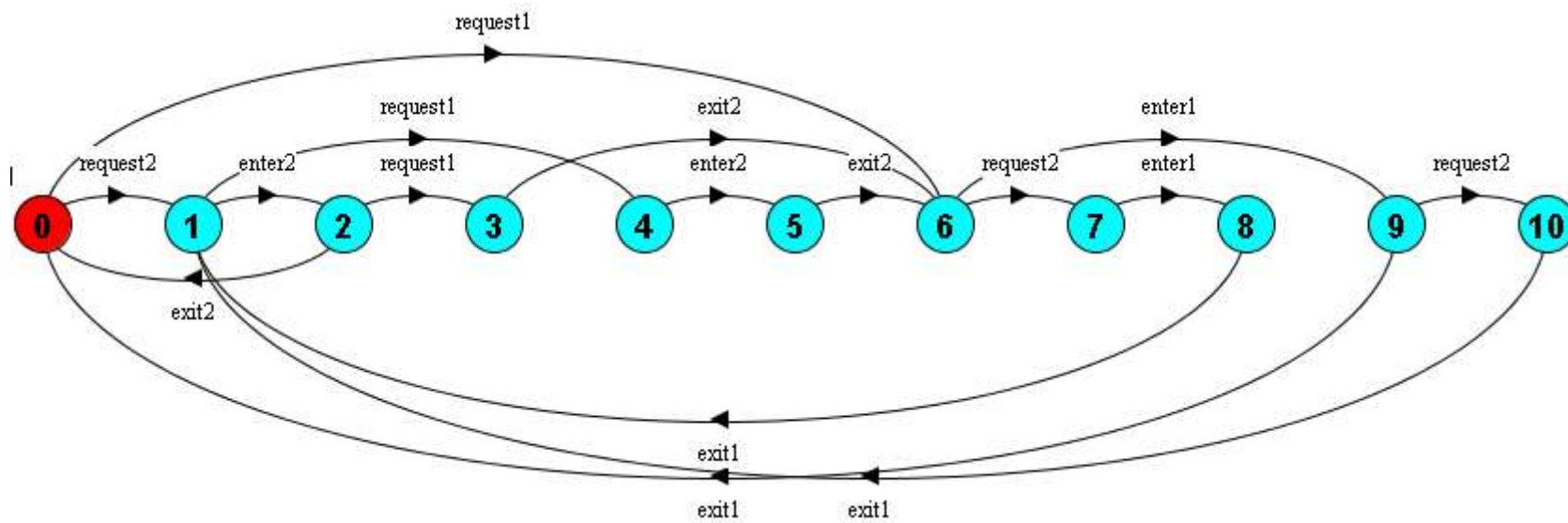
```
CONTROLLER = ( request1 -> TURN1  
               | request2 -> TURN2 ) ,
```

```
TURN1 = ( request2 -> enter1 -> exit1 -> TURN2  
         | enter1 -> ( request2 -> exit1 -> TURN2  
                     | exit1 -> CONTROLLER  
                     )  
         ) ,
```

```
TURN2 = ( request1 -> enter2 -> exit2 -> TURN1  
         | enter2 -> ( request1 -> exit2 -> TURN1  
                     | exit2 -> CONTROLLER  
                     )  
         ) .
```

```
||MUTUAL_EXCLUSION = ( USER1 || USER2 || CONTROLLER ) .
```

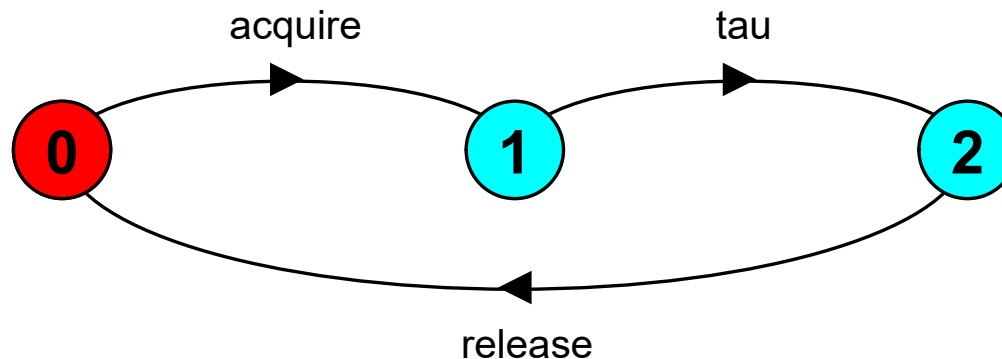

Esempio: mutua esclusione (2)



Hiding

- Con " $P@ \{a_1, \dots, a_x\}$ " si **limita l'alfabeto** di P alle azioni $\{a_1, \dots, a_x\}$
 - **Altre azioni** vengono trasformate in azione locale "**tau**"
 - In questo modo si **evita** che tali azioni vengano **sincronizzate** con altri processi

$USER = (acquire \rightarrow use \rightarrow release \rightarrow USER)$
 $@ \{acquire, release\} .$



Formalmente

Dato un LTS $A = (Q, \Sigma, \delta, q_0)$, l'LTS ottenuto **limitando il suo alfabeto a** $\Sigma' \subseteq \Sigma$ è $A@ \Sigma' = (Q, \Sigma', \delta', q_0)$, dove δ' è definita:

- per l'**azione** speciale **tau** (caso di azioni che **si nascondono**)

$$\delta'(p, \text{tau}) = \bigcup_{a \notin \Sigma'} \delta(p, a)$$

cioè se $p \xrightarrow{a} p' \wedge a \notin \Sigma'$ allora in $A@ \Sigma'$ si ha $p \xrightarrow{\text{tau}} p'$

- per ogni $a \in \Sigma'$ (caso di azioni che **non si nascondono**)

$$\delta'(p, a) = \delta(p, a)$$

cioè se $p \xrightarrow{a} p' \wedge a \in \Sigma'$ allora in $A@ \Sigma'$ si ha $p \xrightarrow{a} p'$

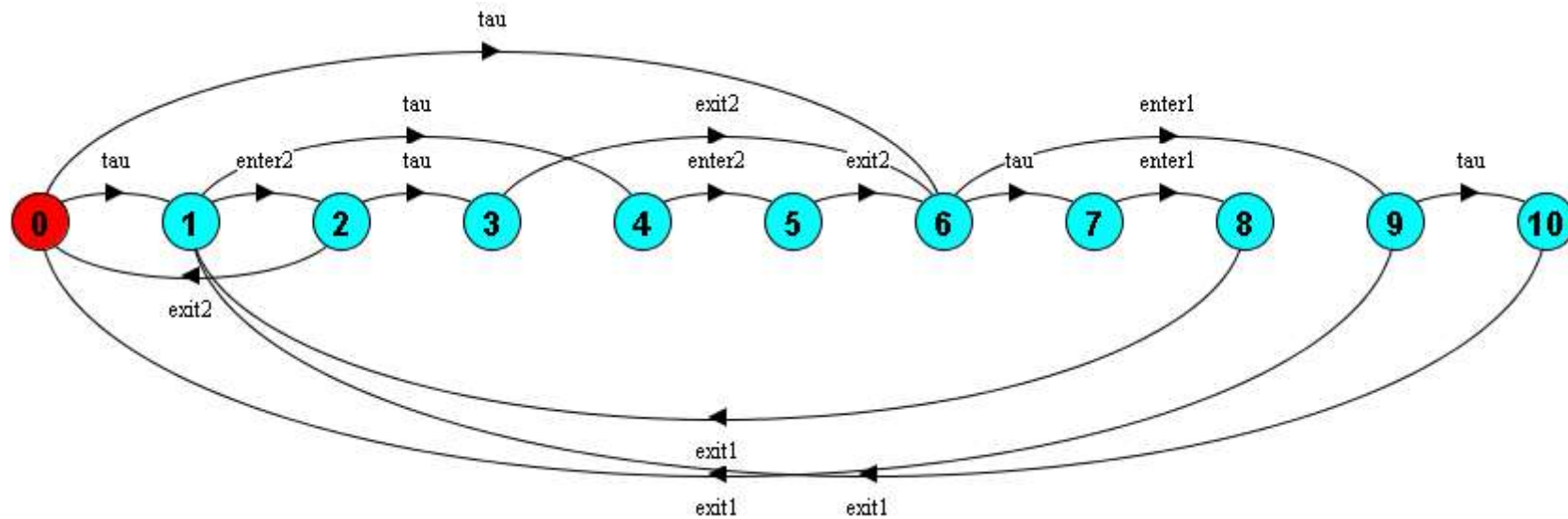
Esempio: mutua esclusione (3)

- Possiamo **nascondere le azioni di richiesta**

```
||MUTUAL_EXCLUSION =  
    ( USER1 || USER2 || CONTROLLER )@{enter1,enter2,exit1,exit2}.
```

- In questo modo possiamo **comporre** l'LTS risultante **con un altro LTS che usa azioni di richiesta** senza che avvenga sincronizzazione

Esempio: mutua esclusione (3)



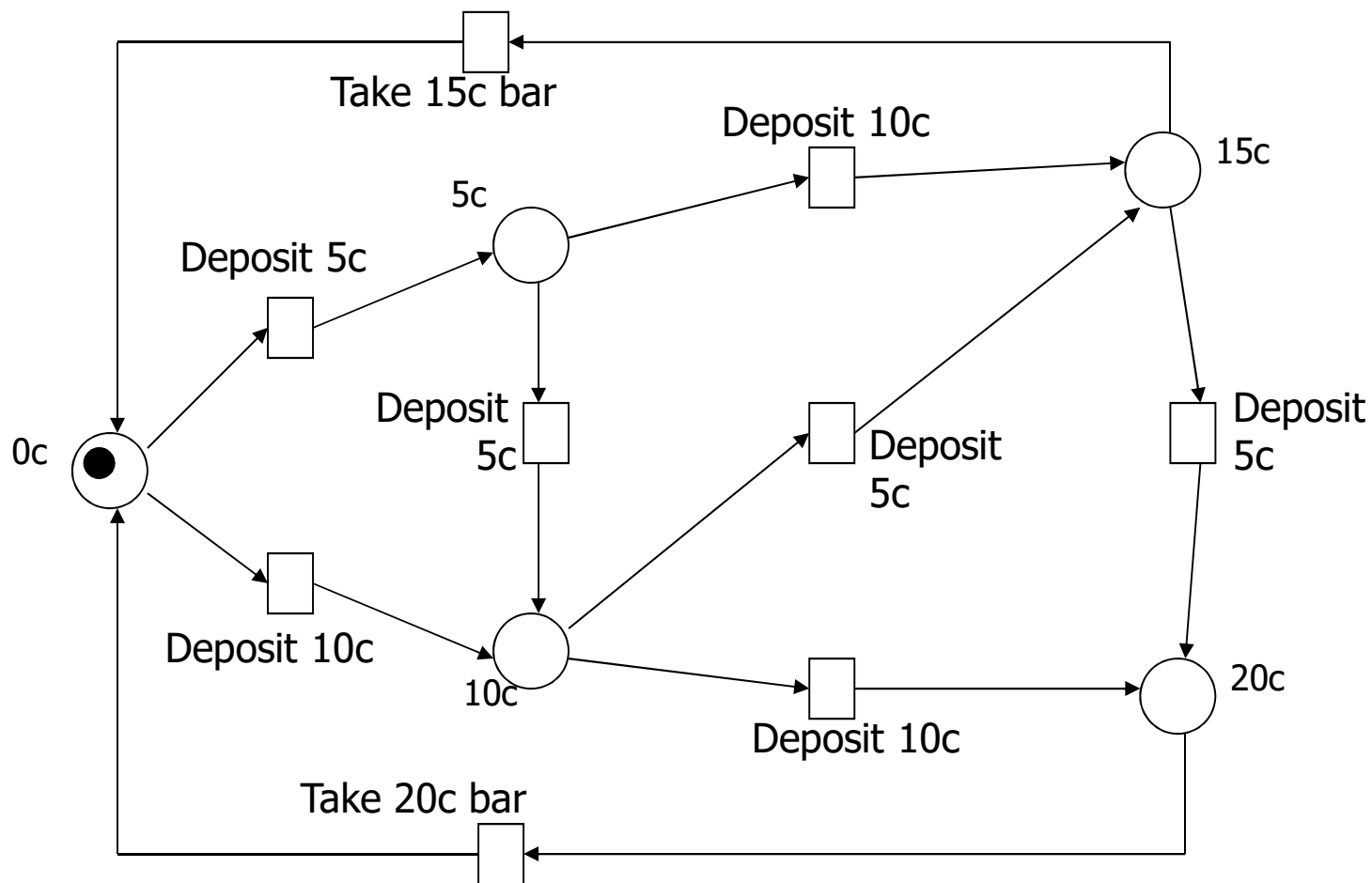
Reti di Petri (Petri nets)

- Approccio “**grafico**” alla rappresentazione dei **sistemi concorrenti**
 - Può essere considerato un’**estensione naturale degli automi** per sistemi concorrenti
 - Gli stati sono detti “**piazze**” (place)
 - Più piazze sono contemporaneamente **attive**
 - indicato dalla presenza di “**token**” al loro interno
 - Le **transizioni** modificano gli stati attivi:
 - **Consumano** un multi-insieme di stati attivi (token)
 - **Generano** un nuovo multi-insieme di stati attivi (token)

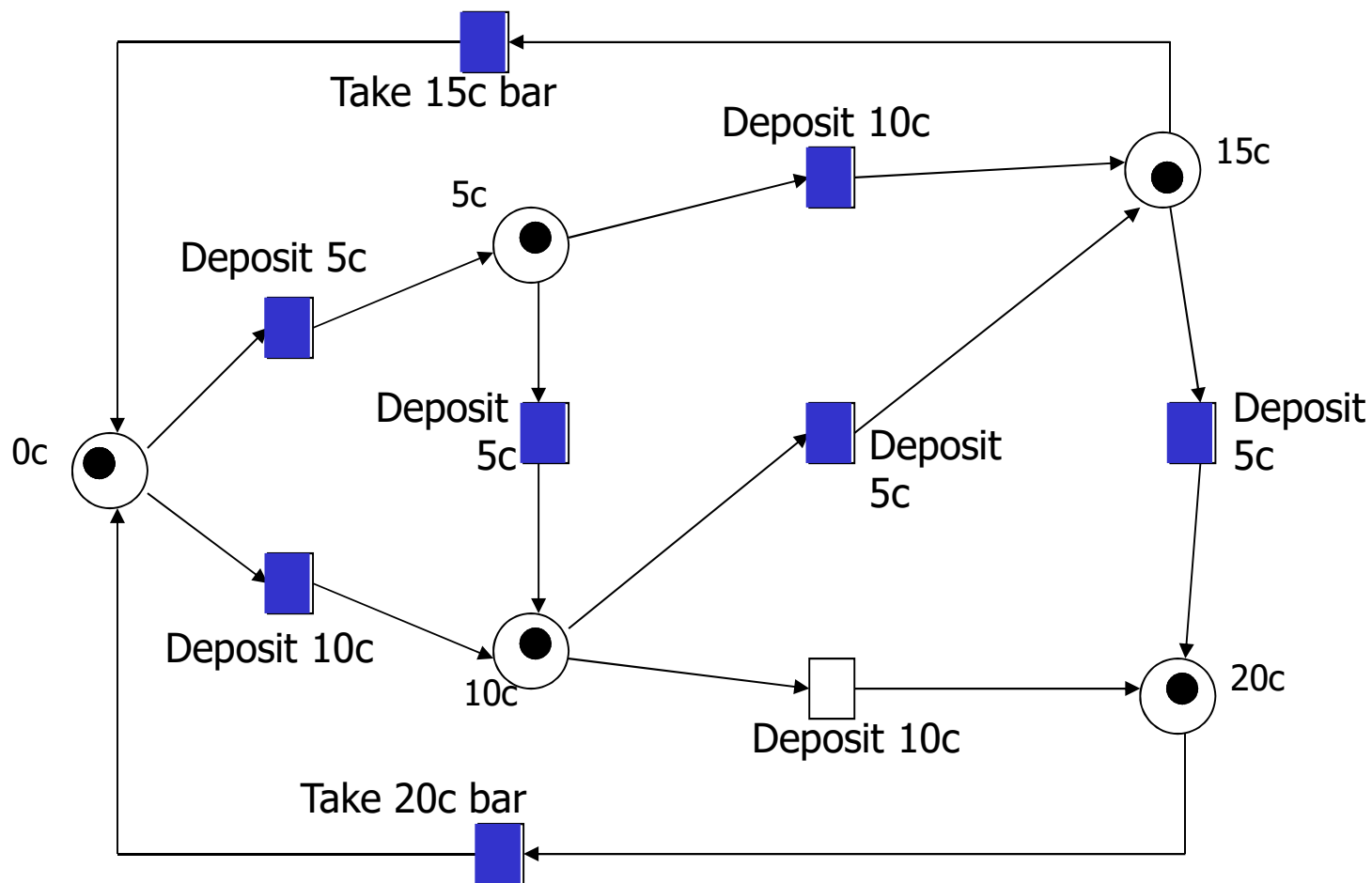
Esempio: distributore automatico

- Il distributore fornisce due tipi di snack, da 15 e 20 centesimi
- Si usano monete da 10 e 5 centesimi
- Il distributore non prevede resto

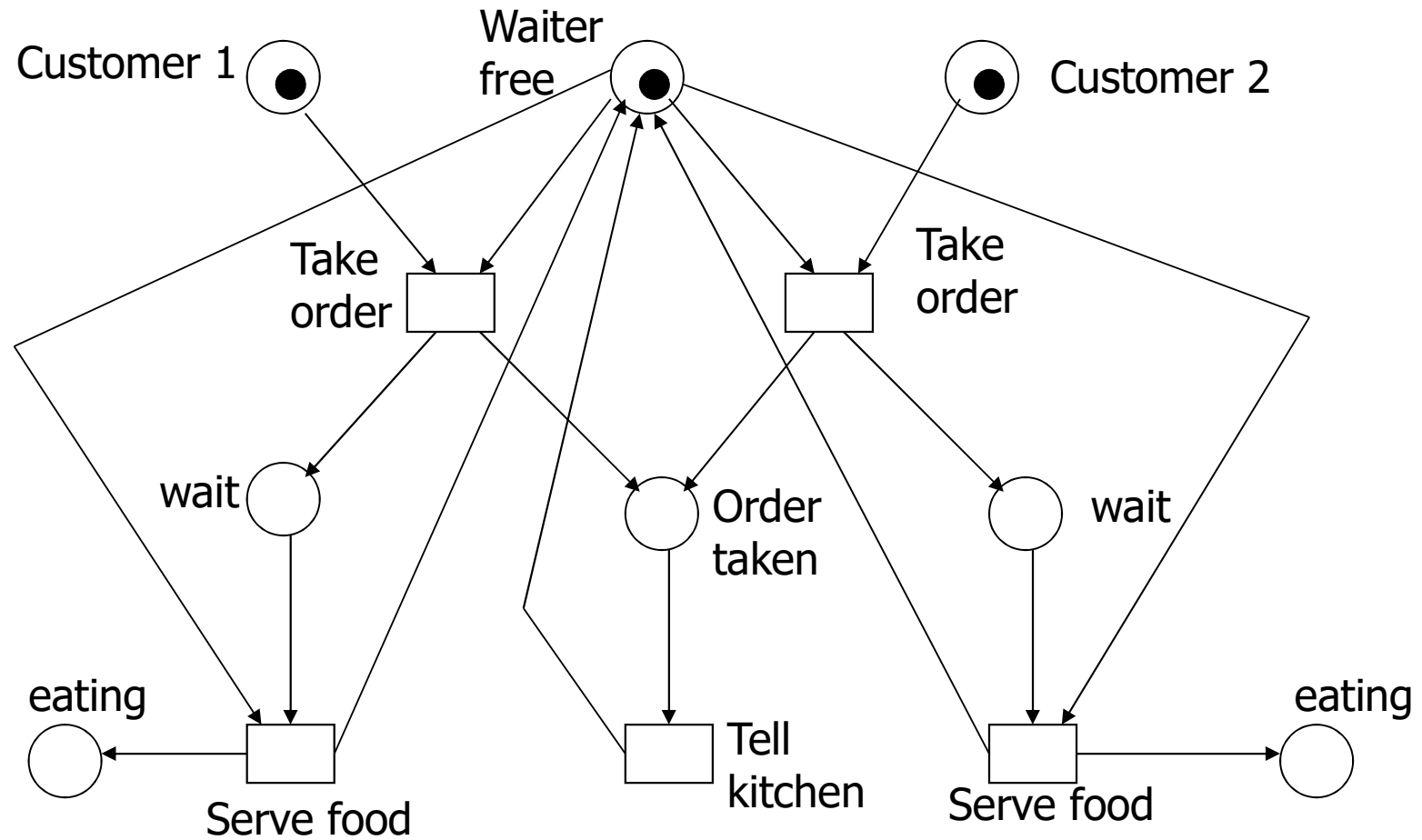
Esempio: distributore automatico



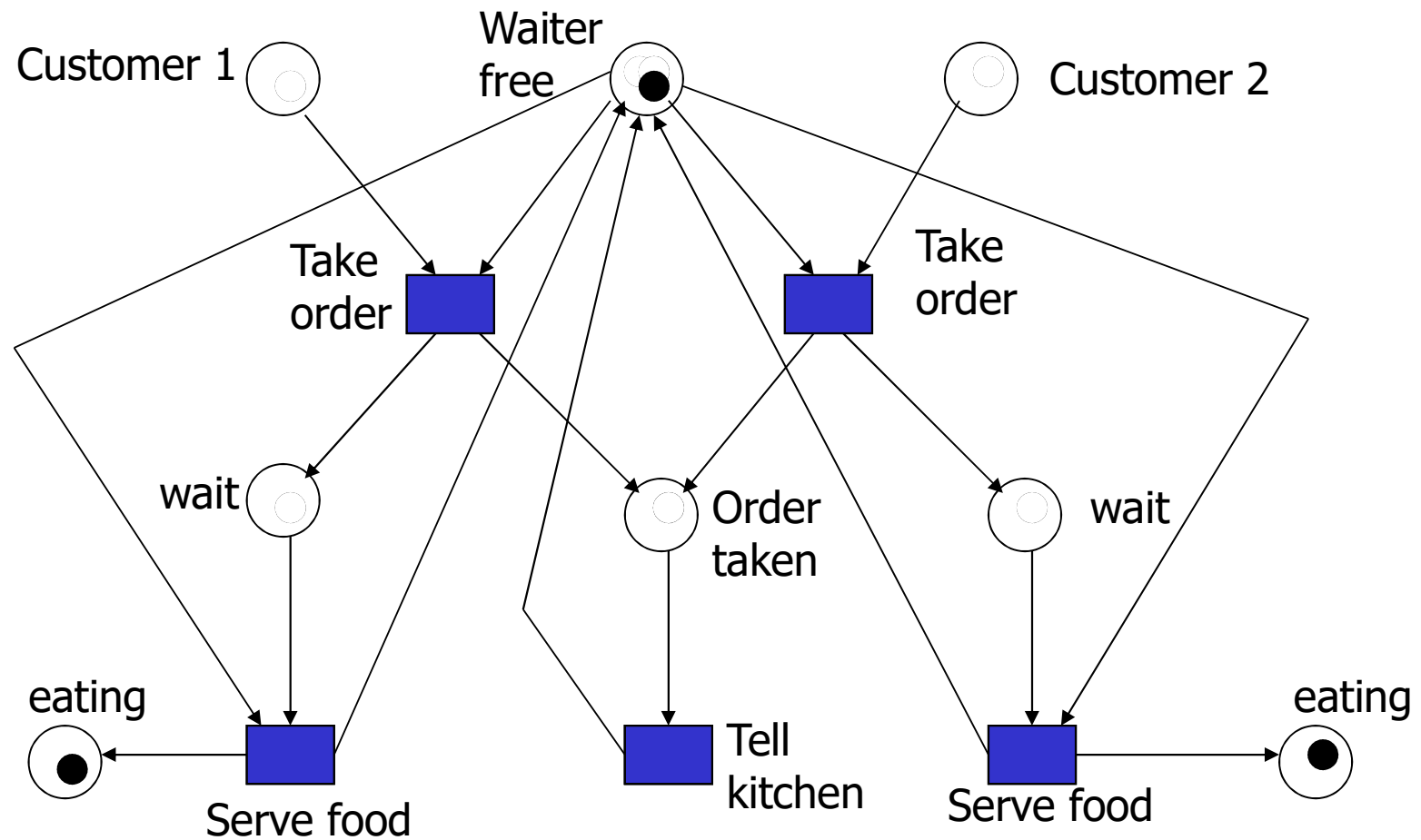
Esempio: distributore automatico



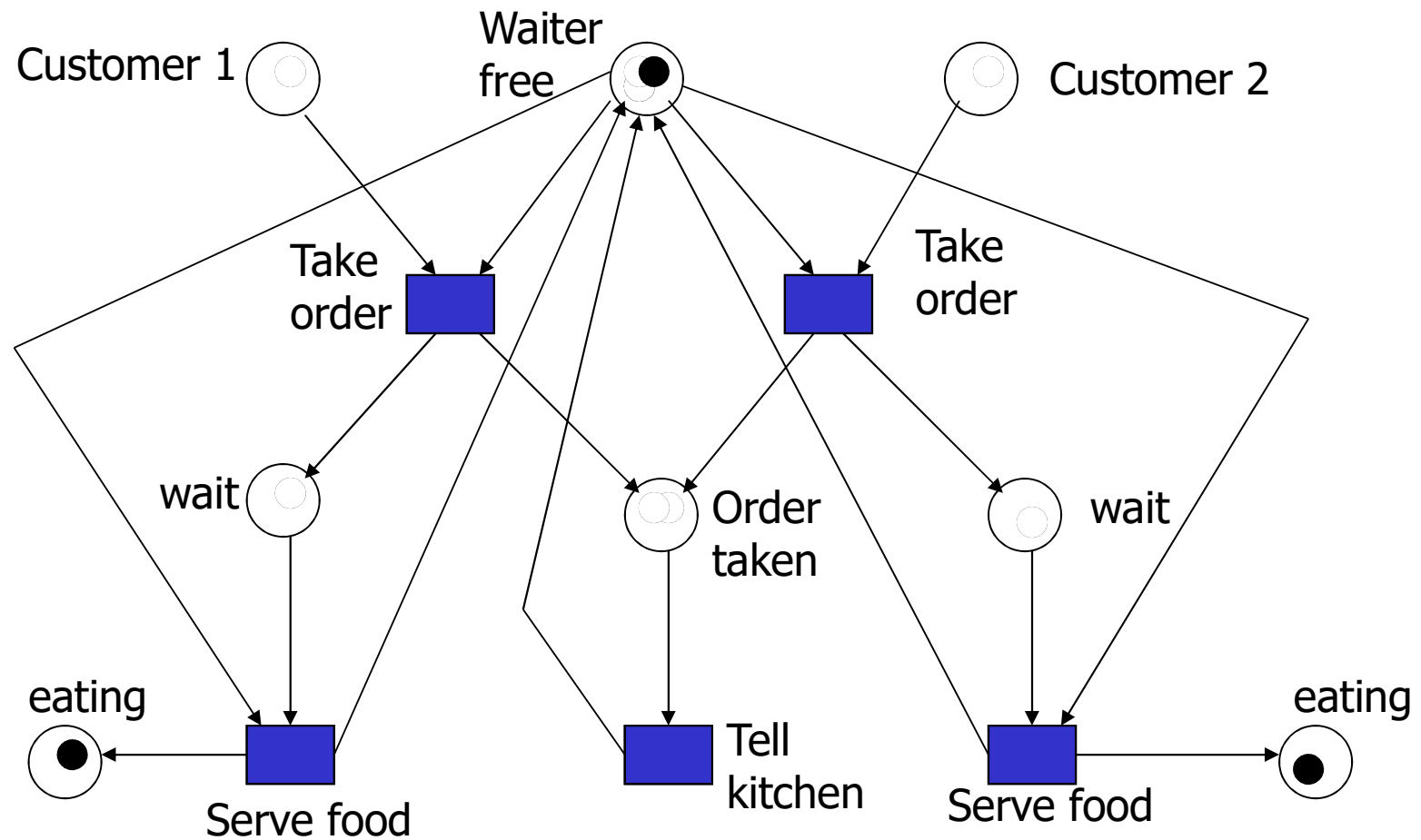
Esempio: ristorante



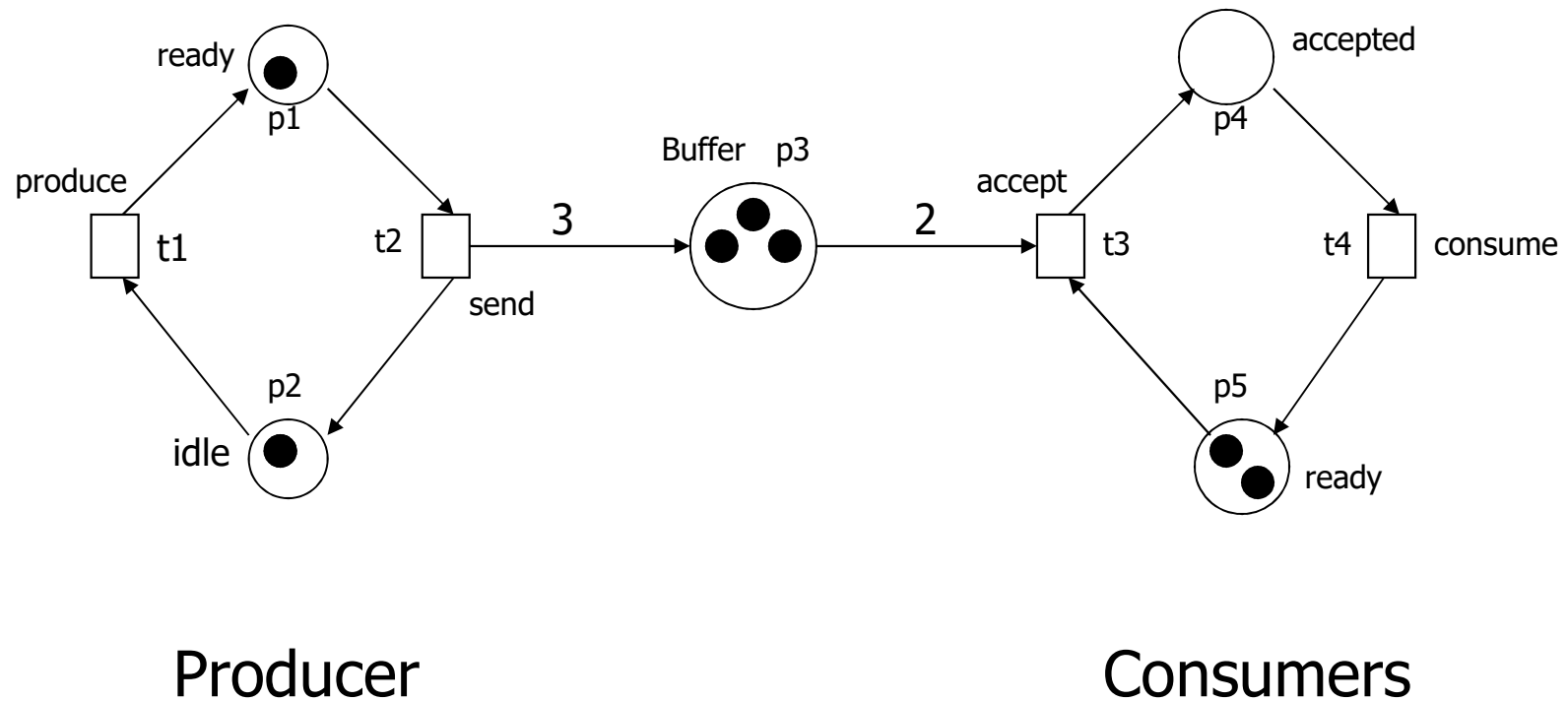
Esempio: ristorante (scenario 1)



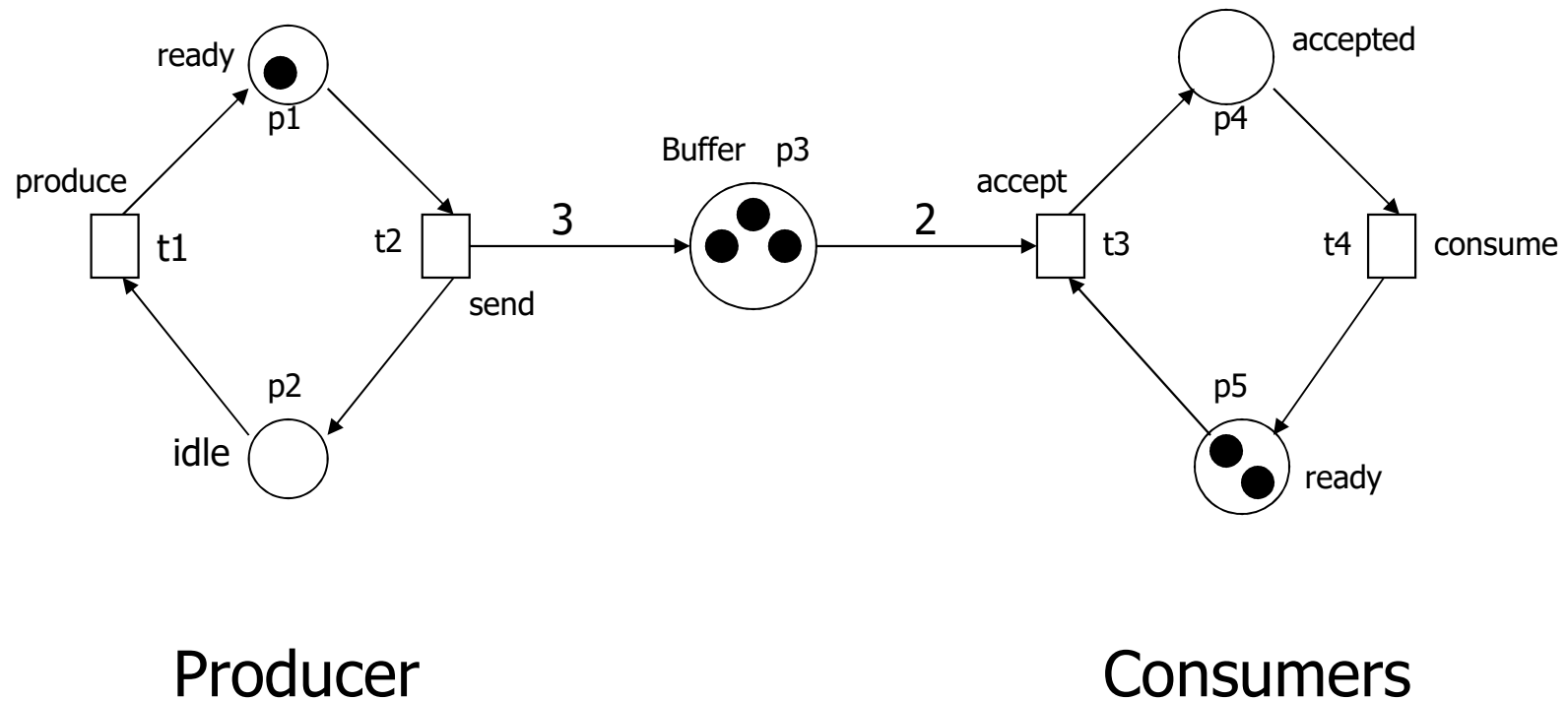
Esempio: ristorante (scenario 2)



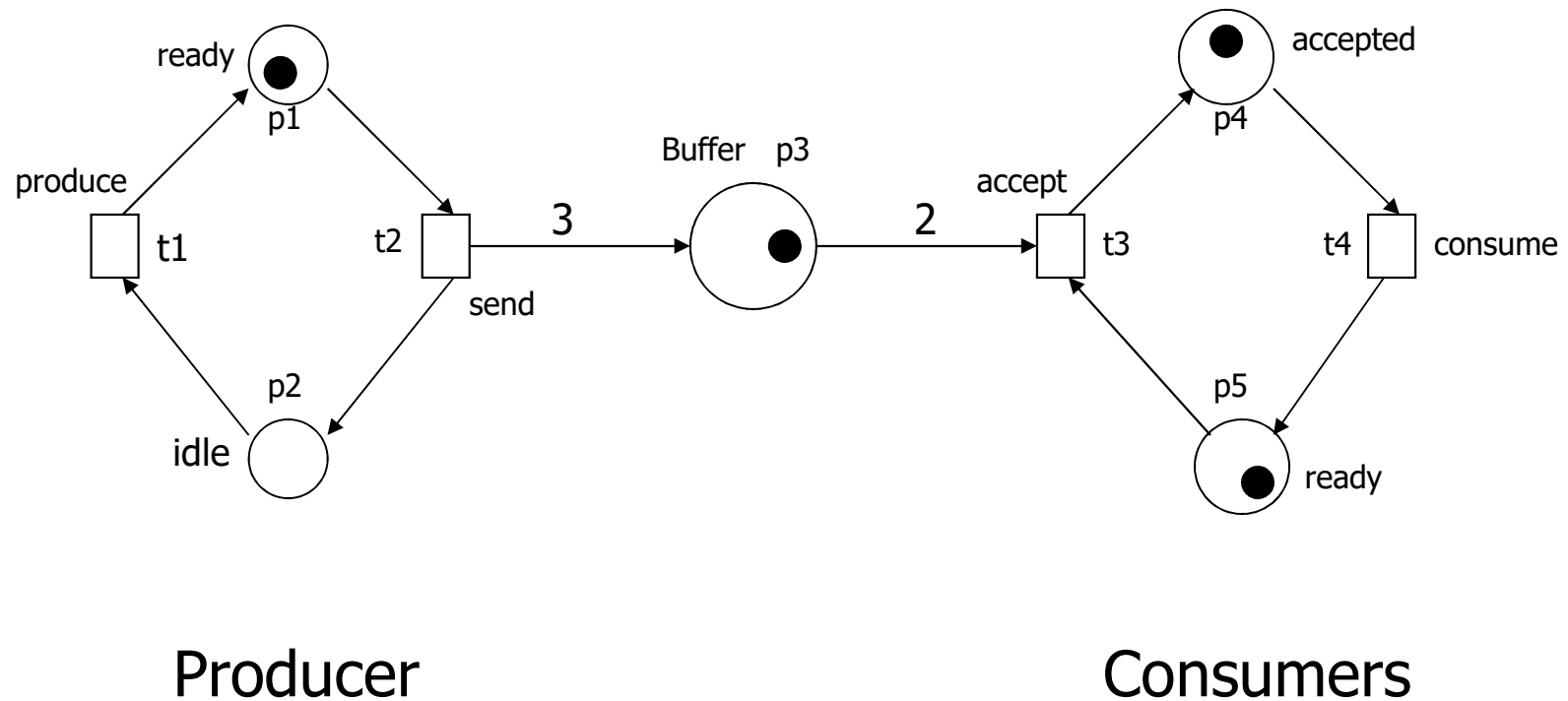
Esempio: produttore / consumatori



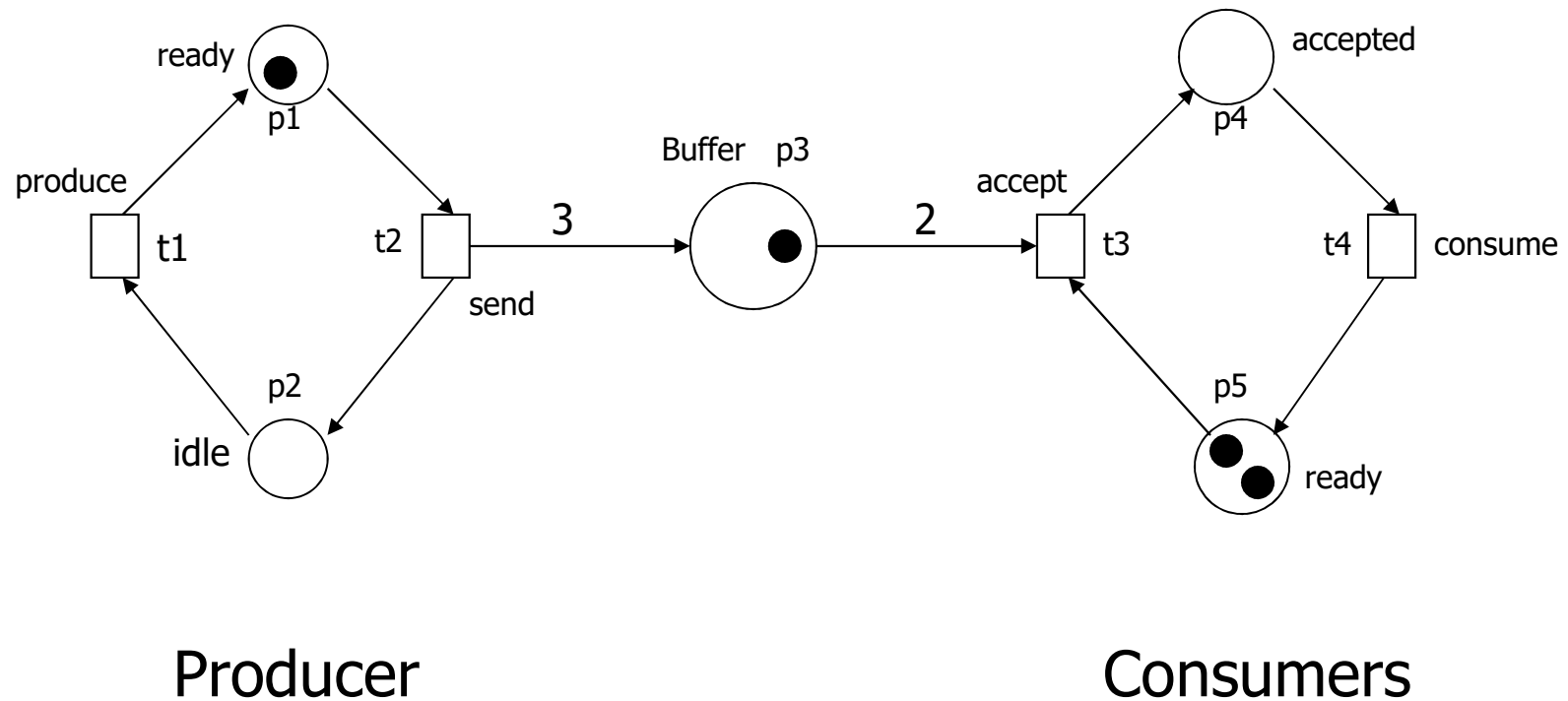
Esempio: produttore / consumatori



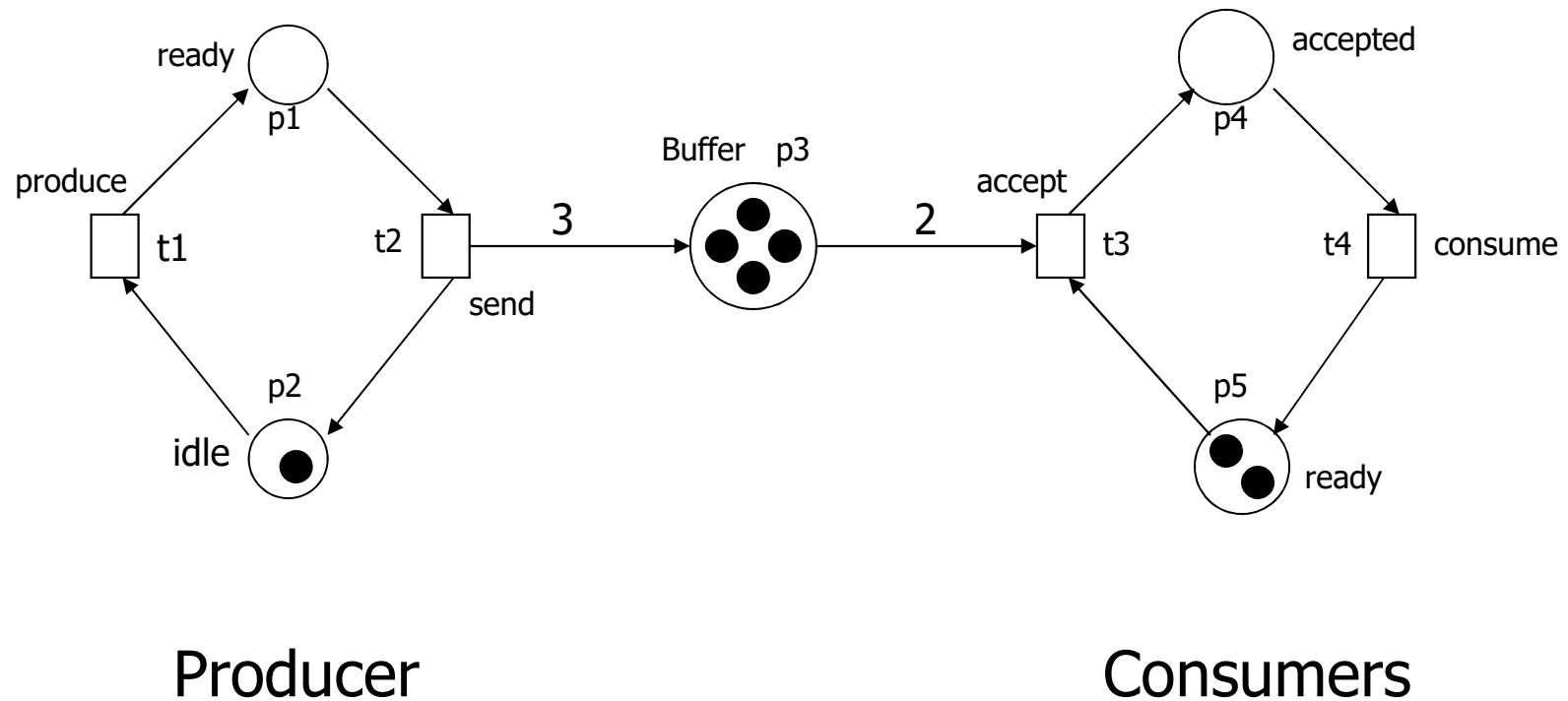
Esempio: produttore / consumatori



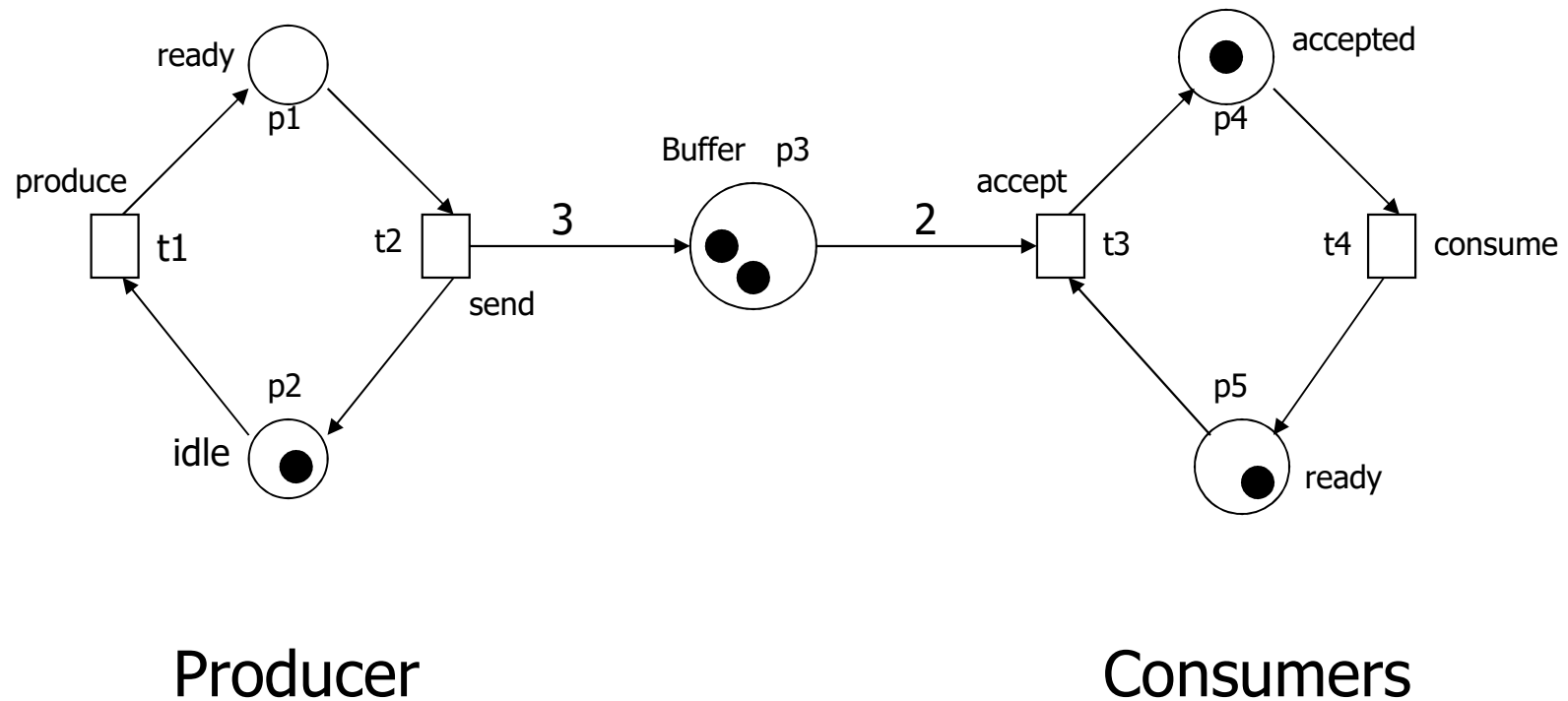
Esempio: produttore / consumatori



Esempio: produttore / consumatori



Esempio: produttore / consumatori



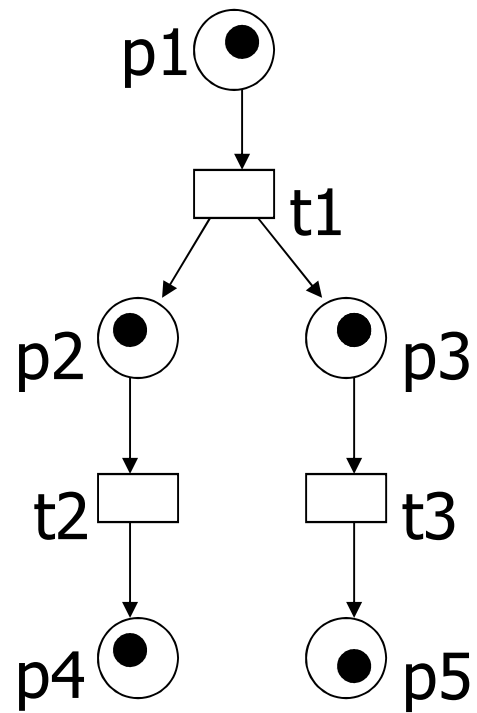
Definizione formale di rete di Petri

- Una **Petri Net** (PN) è una quintupla del seguente tipo:
 - **$PN = (P, T, F, W, M_0)$**
 - $P = \{p_0, p_1, \dots, p_m\}$: a finite set of places
 - $T = \{t_1, t_2, \dots, t_n\}$: a finite set of transitions
 - $F \subseteq (P \times T) \cup (T \times P)$: a set of arcs (flow relation)
 - $W: F \rightarrow \{1, 2, 3, \dots\}$ weight function
 - $M_0: P \rightarrow \{0, 1, 2, \dots\}$ initial marking
 - $P \cap T = \emptyset \quad P \cup T \neq \emptyset$

Marking graph

- Il marking graph di una Petri Net è un **LTS** che ne definisce il comportamento
 - **Configurazione** della PN (chiamato **marking**): un **multi-insieme di piazze** che indica la quantità di token presenti in ciascuna piazza
 - **Passaggio di configurazione**: indica in che modo l'**esecuzione di una transizione** della rete di Petri modifica il marking corrente
 - **Configurazione iniziale**: marking iniziale M_0

Esempio: markings



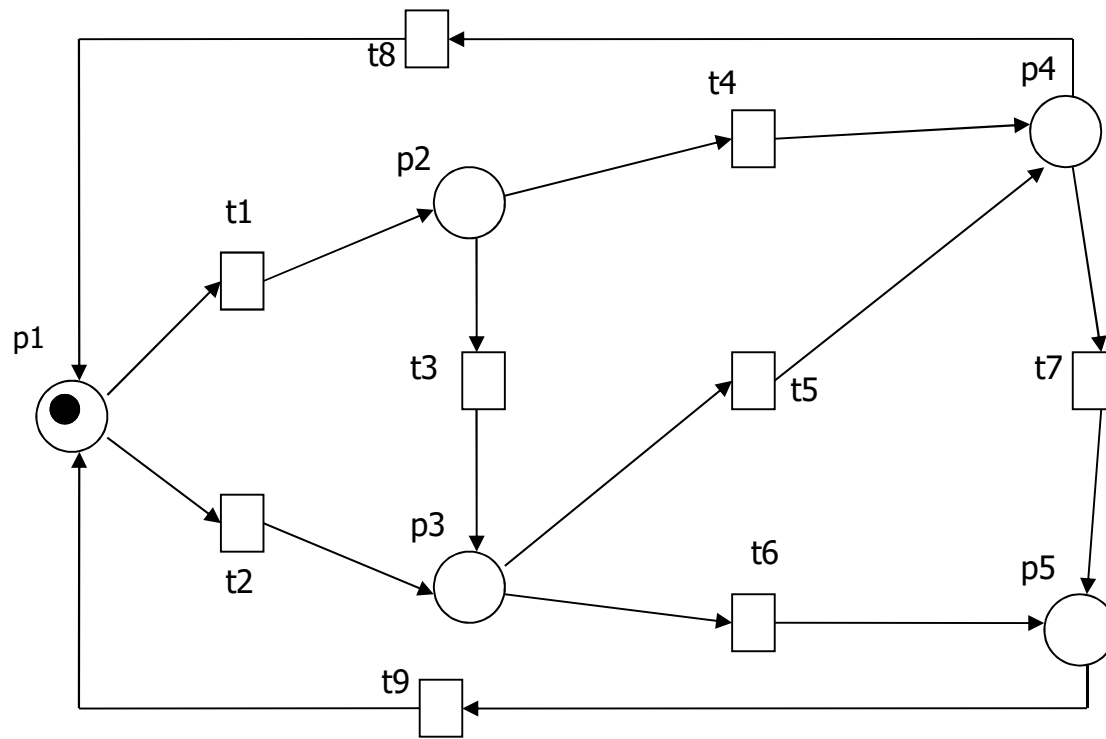
$$M0 = (1, 0, 0, 0, 0)$$

$$M1 = (0, 1, 1, 0, 0)$$

$$M2 = (0, 0, 1, 1, 0)$$

$$M3 = (0, 0, 0, 1, 1)$$

Esempio: marking graph



Marking iniziale: M0

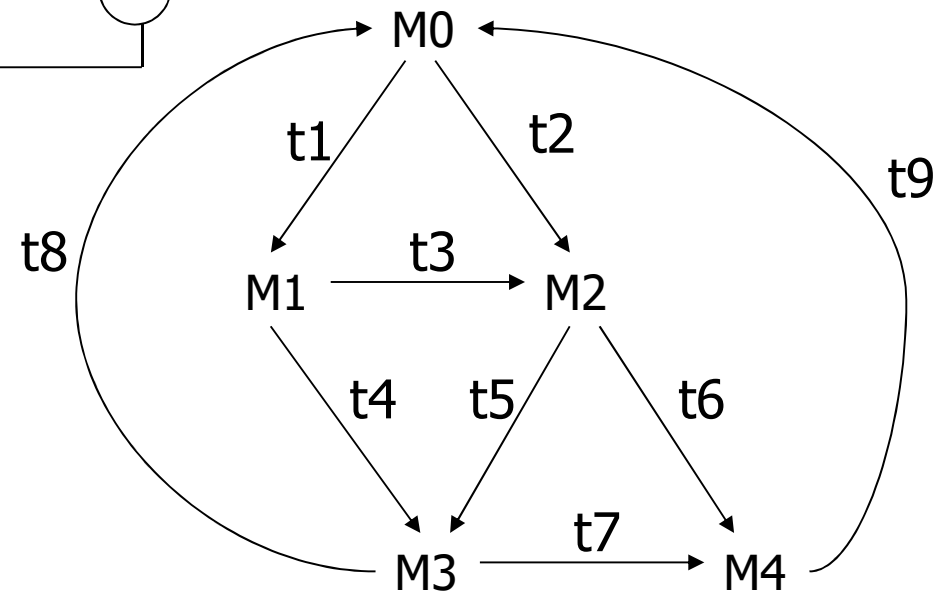
$M0 = (1,0,0,0,0)$

$M1 = (0,1,0,0,0)$

$M2 = (0,0,1,0,0)$

$M3 = (0,0,0,1,0)$

$M4 = (0,0,0,0,1)$



Formalmente

- Un **marking** M è un multi-insieme di piazze:

$$M : P \rightarrow \{0, 1, 2, \dots\}$$

- Una **transizione** $t \in T$ di una PN è **abilitata** in un marking M se:

$$\text{Per ogni } p \in P \quad M(p) \geq W(p, t)$$

- L'**esecuzione** di una $t \in T$ abilitata nel marking M fa **passare** la PN al marking M' dato da:

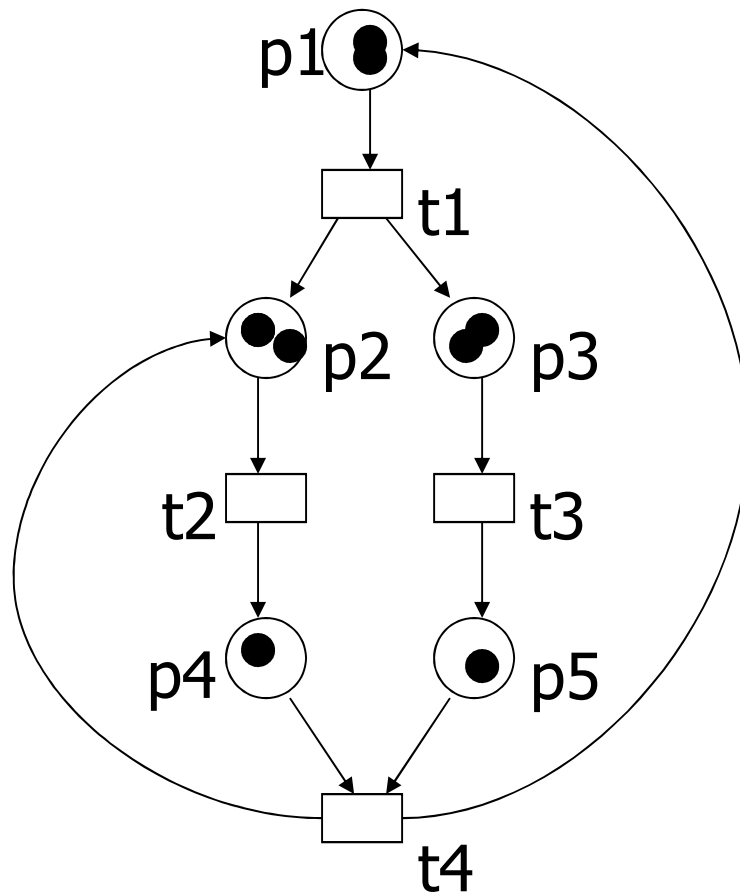
$$\text{Per ogni } p \in P \quad M'(p) = M(p) - W(p, t) + W(t, p)$$

Nota: si assume che il peso W di un arco non in F sia 0

Marking graph (continua)

- A volte il marking graph risulta essere **infinito**:
 - Succede quando la rete di Petri è “unbounded”, cioè può produrre una quantità illimitata di token
 - Esempio: si veda prossima slide

Marking graph (continua)



$$M0 = (1, 0, 0, 0, 0)$$

$$M1 = (0, 1, 1, 0, 0)$$

$$M3 = (0, 0, 0, 1, 1)$$

$$M4 = (1, 1, 0, 0, 0)$$

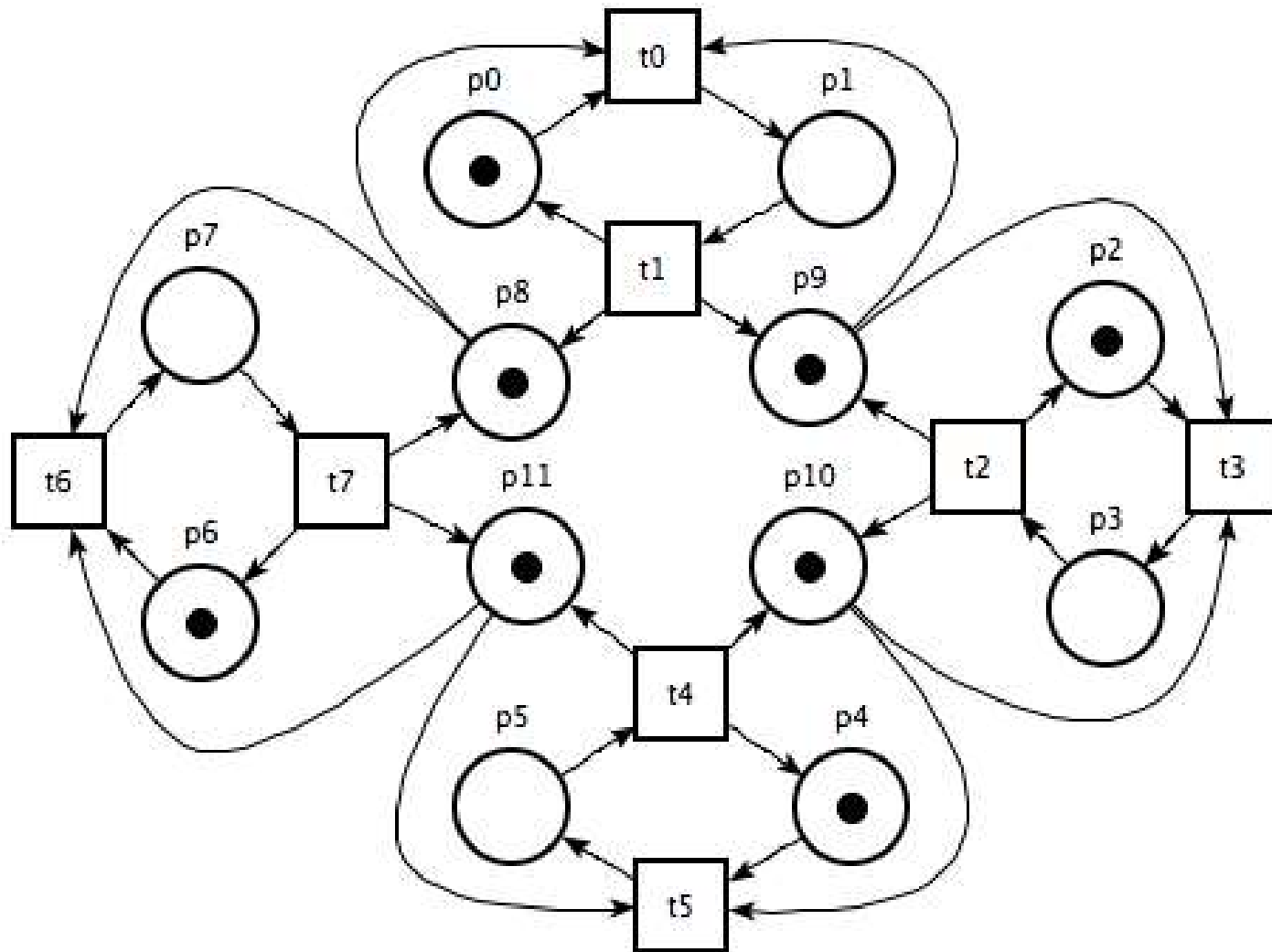
$$M5 = (0, 2, 1, 0, 0)$$

Rete di Petri unbounded

Model checking

- Su Petri nets “bounded” si verifica se una **formula** di LTL è **soddisfatta** come già visto:
 - Si considera il **marking graph** come un **LTS** che rappresenta il comportamento della rete di Petri
 - E poi i **modelli** π di tutte le sue tracce massimali
- Ora però **proposizioni** da **stati** dell’LTS:
 - In questo caso gli stati sono i **marking** della PN
 - Sono definite proposizioni che consentono di osservare le piazze: **proposizione “p”** vera se e solo se **piazza p contiene almeno un token**

Esempio: 4 dining philosopher



Esempio: 4 dining philosopher

- Alcune proprietà esprimibili in LTL (alcune vere, alcune false – usare TINA per controllare):
 - $[](-p1 \wedge p3)$
due filosofi vicini non mangiano contemporaneamente
 - $[]((-p8) \Rightarrow (<>p8))$
una forchetta in uso, verrà rilasciata
 - $<>p1$
il primo filosofo mangerà
 - $\neg(<>(p1 \wedge p5))$
non si raggiunge uno stato in cui due filosofi lontani mangiano insieme

